CS2019 – Databases and Data Management

Assessment 2 – Arts Museum Database

Worked reference solution to Tasks 1–10

Target platform: MySQL 8.0.16 or later (InnoDB) · Prepared November 2026

> **About this document.** Every answer is labelled with its task number. The accompanying files are: `artsmuseum.sql` (creates the database, loads the sample data, installs views and triggers), `04_transactions.sql` (the thirteen transaction requirements) and the (E)ER diagram as PNG and SVG. The relational schemas in Task 4, the SQL in Tasks 8–9 and the query results in Appendix A are generated from those same files, so the report and the database always agree with each other.

Contents

Task	Content	Marks
1	Requirements analysis: issues found, refined data and transaction requirements	3
2	(E)ER diagram in UML notation, assumptions and design decisions	10
3	Relational mapping: the top challenge and assumptions	2
4	Normalised relational schemas (3NF) with functional-dependency analysis	10
5	Why the compound-key table Lending is in 3NF	3
6	Modelling a class/subclass relationship: ArtObject and Painting	3
7	How the design fulfils the requirements (for the museum owner)	4
8	MySQL implementation: CREATE TABLE statements, data types and integrity features	6
9	Sample data: INSERT INTO statements	6
10	Top three lessons learnt	3
A, B	Appendix A: transaction SQL with sample output. Appendix B: references	–

Task 1 – Requirements analysis

1.1 Approach

We read the brief sentence by sentence and put four questions to each one: *Is it ambiguous? Is something missing that the design will need? Do two statements disagree? Can every transaction requirement be answered from the data that is described?* Each finding was settled by a decision. The first pass found the obvious gaps (galleries, invitations, replicas, outgoing loans); a second pass, made after sketching the EER diagram, found the subtler ones (the artist key that cannot contain a missing death date, the overloaded word "type", status values that duplicate gallery placement). The table below lists every finding with the decision taken and the reason for it. Section 1.3 then gives the final data requirements, and section 1.4 the final transaction requirements.

1.2 Issues found and decisions taken

#	Where in the brief	Problem	Decision and rationale
R1	"type" (five different uses)	The word means the object category (painting, sculpture, statue, others), the painting type (oil, watercolour), the type of an "other" object (print, photo), the kind of staff, and the exhibition feature "type". One word for five things would produce clashing column names and makes "count per type" ambiguous.	Use distinct names: **category**, **paintingMedium**, **otherType**, **staffType**, **featureType**. "Per type" in the transaction list means per category.
R2	"painting, sculpture and statue"	Sculpture and statue have exactly the same attributes (material, height, weight, style) and nothing says how to tell them apart, so the same object could be classified either way.	Keep both, because the brief counts and exhibits them separately. Define: a **statue** is a free-standing figure of a person, animal or deity; every other carved, cast or modelled three-dimensional work is a **sculpture**. Their shared attributes are factored into one abstract generalisation.
R3	"in any style such as…" repeated for painting, sculpture/statue and others	The same attribute is listed in every category, so it belongs to all art objects.	Record **style once, on the art object** (a common attribute is placed in the superclass).
R4	"other museums can also borrow art objects from this museum"	Lending is mentioned, and the status "on loan" implies it, but data is specified only for incoming loans.	Add **outgoing loan data**: borrowing museum, date lent, date due back, date returned.
R5	"date to be returned"	With only a due date the museum cannot tell whether an object is still held, and an object borrowed twice would lose its first loan.	Add the **actual return date** and keep **every loan as its own record**, so loan history is preserved.
R6	"status" recorded only for permanent objects; list ends "etc."	Borrowed objects also move between display, store and restoration, and the regular exhibition shows borrowed objects. "On display" duplicates the gallery an object hangs in.	Record status for **every** object with the values ON_DISPLAY, IN_STORE, UNDER_RESTORATION, ON_LOAN (permanent objects only) and RETURNED (borrowed objects only). An object is ON_DISPLAY **exactly when** it is placed in a gallery.
R7	"The name, date born and date died uniquely identify an artist"; artists recorded "if known"	A living artist has no death date, and a missing value cannot be part of a key. "If known" also means some objects have no artist at all.	The artist of an object is **optional**. An artist record needs a name and a birth date; the death date is empty for living artists. The identity rule must treat "no death date" as a value, so two living artists with the same name and birth date are still rejected.
R8	"country or culture of its origin" and "epoch" recorded for objects **and** for artists	An artist's origin can differ from the origin of an object, and "country or culture" mixes two notions (Italy, Italian). Free text would allow "Italian", "italian" and "Italy".	Keep both (the object's is the one displayed). Use **one controlled list** each for origin ("country or culture"), epoch, style and material.
R9	"AOXXXX", "LMYYYY"	The characters are not defined; four characters allow only 10,000 objects if they are digits, and YYYY might even be a year.	Interpret both as **two letters plus four digits** (AO0001, LM1001), enforced by a rule. We flag the 10,000-object ceiling: the pattern must be widened before the collection approaches it.
R10	cost, height, weight, "year of its creation"	No units or currency; ancient objects have years before the common era, and some years are unknown.	Cost in pounds sterling, height in cm, weight in kg; creation year is a whole number (negative = BCE) and may be **unknown**.
R11	"permanent collection" and "borrowed"	The brief does not say whether an object can be both or neither.	Every object is **exactly one** of the two. A borrowed object that is later bought is registered as a new permanent object.

#	Where in the brief	Problem	Decision and rationale
R12	"regular galleries", "any gallery"	Galleries are used but never described.	Add **Gallery** (name, floor level). Each gallery is part of exactly one regular exhibition.
R13	"exhibitions that are regular … and featured"	No exhibition attributes are given. A regular exhibition has no end; a featured one occurs "only for specific periods".	Every exhibition has a name and description. A featured exhibition **must** have a start and an end date (end not before start) and one or more **features** (artist, category, style, epoch or origin).
R14	"involving all the available art objects in regular galleries" / "involving art objects sharing a feature"	Nothing says how membership is recorded.	Regular exhibition: an object **is placed in a gallery**. Featured exhibition: the included objects are **listed explicitly**.
R15	"This information is displayed … in all the exhibitions" versus "this information is not displayed"	Two different groups of attributes are meant: title, artist, year, description, origin, epoch are shown; acquisition and loan details are not.	Treat the second group as **internal** and expose only the first through a **view**.
R16	"the museum sells replicas … an important source of income"	Replicas are mentioned but no data is described.	Record, per featured exhibition and per included object, the **replica price and the number sold**. A replica can only be sold for an object of that exhibition.
R17	"loyal visitor … preferences for featured exhibitions"	Only the loyalty number is given; "preferences" is undefined and there is no contact data to send invitations to.	Add name, address, email, phone and join date. A **preference** is a feature the visitor likes, using the same vocabulary as exhibition features (artist, category, style, epoch, origin).
R18	"sends out invitations to loyal visitors"	The invitations themselves are not recorded, so "their loyal visitors" in the transaction list has no meaning.	Record each **invitation** (visitor, exhibition, date sent, channel, response). "Their loyal visitors" = **the visitors invited** to that exhibition.
R19	"several types of staff, such as curators, gallery supervisors, etc."	No staff attributes; "etc." means other kinds of staff exist.	Staff have an identifier, name, job title, contact details and hire date. Only curators and supervisors have extra data and relationships; other staff are simply staff.
R20	"certain number (say 20) and type of art objects" / "several curators … certain large art objects"	Is 20 a limit or a target? Is "type" the object category? Can a curator have several types?	The number is a **maximum** (default 20, set per curator); the type is **one specialisation category**; a curator may only maintain objects of that category; an object may have **several curators**.
R21	"allocated to any gallery in a regular exhibition or to a featured exhibition"	Over what period? Can a supervisor have several allocations?	Two allocation relationships: supervisor–gallery (from date, optional to date) and supervisor–featured exhibition (allocation date); a supervisor may have any number over time.
R22	Transaction wording	"count per type", "sort the list", "their loyal visitors", "borrowed art objects" can each be read in several ways.	Fixed in section 1.4.

1.3 Final data requirements

Changes to the original brief are tagged with the issue number, for example *[R4]*.

A. The museum and the Arts Friends Circle

- **DR1.** The museum owns a permanent collection of art objects and also borrows objects from other museums for a limited time. Other museums may borrow objects from this museum. The museums that share objects this way form the Arts Friends Circle.
- **DR2.** For every other member of the circle the museum records its name (unique), description, address, phone number and the name of the current contact.
- **DR3.** Every borrowing (into this museum) and every lending (out of this museum) is recorded as a separate loan, so that an object can be loaned many times and its loan history is kept. A borrowing records the lending museum, the date borrowed, the date due back and, once it has happened, the date returned. A lending records the borrowing museum, the date lent, the date due back and the date returned. *[R4, R5]*

B. Art objects

- **DR4.** Each art object has a unique identifier made of the letters AO and four digits, a title, a description, a year of creation (a whole number, negative for BCE, possibly unknown) and an origin ("country or culture"), an epoch and a style, each chosen from a controlled list. *[R3, R8, R9, R10]*
- **DR5.** An art object has at most one artist; for some objects the artist is unknown. *[R7]*
- **DR6.** Every art object belongs to exactly one category: painting, sculpture, statue or other. A statue is a free-standing figure of a person, animal or deity; any other three-dimensional work is a sculpture; everything that is neither painting, sculpture nor statue is an "other" object. *[R1, R2]*
- **DR7.** A painting has a painting medium (oil, watercolour, acrylic …) and, if known, a surface material (paper, canvas, wood …).
- **DR8.** A sculpture or a statue has a material (wood, stone, bronze …), a height in centimetres and a weight in kilograms.
- **DR9.** An "other" object has a type (print, photograph, drawing …). *[R1]*
- **DR10.** Every art object is either a permanent object or a borrowed object, never both. A permanent object has an acquisition date and an acquisition cost in pounds sterling. A borrowed object has one or more loans (DR3). *[R10, R11]*
- **DR11.** Every art object has a status: on display, in store, under restoration, on loan (permanent objects only) or returned (borrowed objects only). An object is on display exactly when it is placed in a gallery. *[R6]*
- **DR12.** The title, artist, year, description, origin and epoch of an object are displayed with it in every exhibition. Acquisition and loan details are never displayed. *[R15]*

C. Artists

- **DR13.** When known, the museum records for an artist the name, date born, date died (absent for a living artist), origin, epoch, main style and a description. The name, date born and date died identify an artist; an artist record needs at least a name and a date born. *[R7, R8]*

D. Exhibitions and galleries

- **DR14.** There are two kinds of exhibition, regular and featured. Every exhibition has a name and a description. *[R13]*
- **DR15.** A regular exhibition is permanent and consists of one or more galleries. A gallery has a name and a floor level and belongs to exactly one regular exhibition. An object on display is placed in one gallery. *[R12, R14]*
- **DR16.** A featured exhibition runs for a specific period (start date and end date, the end not before the start) and is built around one or more features: an artist, a category, a style, an epoch or an origin. The art objects included in it are recorded. *[R13, R14]*
- **DR17.** During a featured exhibition replicas of its objects are sold. For each object the museum records the replica price and the number of replicas sold, and only for objects of that exhibition. *[R16]*

E. Loyal visitors

- **DR18.** A loyal visitor has a unique loyalty number (the letters LM and four digits), first and last name, address, join date and optionally an email address and a phone number. *[R9, R17]*
- **DR19.** A loyal visitor has zero or more preferences; a preference is a feature (artist, category, style, epoch or origin) the visitor likes. *[R17]*
- **DR20.** The museum invites loyal visitors to featured exhibitions. An invitation records the visitor, the exhibition, the date sent, the channel (email, post or SMS) and the response (pending, accepted or declined). The loyal visitors of an exhibition are those invited to it. *[R18]*

F. Staff

- **DR21.** Every staff member has an identifier, first and last name, job title, email address, hire date, an optional phone number and a staff type: curator, supervisor or other. *[R19]*
- **DR22.** A curator has one specialisation (an object category) and a maximum number of objects to maintain (20 unless agreed otherwise). A curator maintains only objects of the specialisation and never more than the maximum. An object may be maintained by several curators. *[R20]*
- **DR23.** A supervisor may be allocated to galleries (from a date, optionally to a date) and to featured exhibitions (from an allocation date), any number of each over time. *[R21]*

1.4 Final transaction requirements

#	Transaction	Change
T1	Add a new art object to the permanent collection. The object, its category details and its permanent-collection details are stored together as one database transaction.	Clarified
T2	Change the end date of a featured exhibition (the new end date may not be before the start date).	Clarified
T3	List all the borrowed art objects with their lender, dates and whether they have been returned.	Clarified (all loans, state shown)
T4	Count the number of art objects created by a given artist (identified by name and date born).	Clarified
T5	Count the number of art objects per category displayed in the regular exhibition; sort with the largest count first, ties in alphabetical order.	"type" becomes category; sort order defined
T6	List all the art objects borrowed from a given Arts Friends Circle museum.	Unchanged
T7	List the staff supervising a given featured exhibition.	Unchanged
T8	List all the curators responsible for maintaining a given art object.	Unchanged
T9	List all featured exhibitions with their start and end dates and their loyal visitors (the visitors invited); exhibitions without invitations are still listed.	Clarified
T10	List all the art objects used in a given featured exhibition that a given artist created.	Unchanged
T11	List the permanent objects that are lent out and not yet returned.	**New**: needed because outgoing loans are now recorded
T12	Total replicas sold and replica income per featured exhibition.	**New**: replica sales are now recorded
T13	List the loyal visitors whose preferences match a featured exhibition's features and who have not yet been invited.	**New**: puts preferences and invitations to use

1.5 What we kept as given, and why

We kept the four object categories and their attributes (the transaction list and the exhibition features depend on them); the split into permanent and borrowed objects with the details the brief names; time-limited featured exhibitions; the natural identity of an artist (name, date born, date died), which survives as an alternate key; the unique museum name; the AO and LM identifier formats; and the roles of curators and supervisors. Each of these is consistent, is used by at least one transaction, and contains no ambiguity that would change the design, so rewriting it would only add noise. Every transaction requirement was re-checked against the refined data requirements: each can be answered from the data described in section 1.3.

Task 2 – (E)ER diagram

The diagram uses UML class notation. The full diagram is Figure 1; Figures 2 to 5 are enlargements of its four areas so that every label can be read on paper. The same diagram is supplied as `eer-diagram.png` (high resolution) and `eer-diagram.svg` (vector) in the repository.

How to read it. A box is an entity (class); italic names are abstract. A hollow triangle is a generalisation with its constraint in braces ({disjoint, complete}: a superclass instance belongs to exactly one subclass and to at least one). A hollow diamond is an aggregation, a filled diamond a composition. A folded-corner note carries a design decision for whoever performs the relational design. Attributes of a relationship sit in a yellow association-class box joined to the relationship by a dashed line. The multiplicity written next to a class says how many instances of that class are linked to **one** instance of the class at the other end. `{id}` marks an identifier, `[0..1]` an optional attribute, «`lookup`» an attribute whose values come from a controlled list.

2.1 Entities, hierarchies and relationships at a glance

Construct	What the diagram shows
Generalisation 1 – ArtObject by category	ArtObject → Painting, ThreeDimensionalWork (abstract: Sculpture, Statue), OtherObject. {disjoint, complete}; "other" is the catch-all of the brief. A two-level hierarchy.
Generalisation 2 – ArtObject by ownership	ArtObject → PermanentObject, BorrowedObject. {disjoint, complete}. Independent of generalisation 1: an object has one category **and** one ownership.
Generalisation 3 – Exhibition	Exhibition (abstract) → RegularExhibition, FeaturedExhibition. {disjoint, complete}.
Generalisation 4 – Staff	Staff → Curator, Supervisor. {disjoint, incomplete}: other staff exist that need no subclass.
Aggregation	RegularExhibition (the whole, hollow diamond) – Gallery (the part): a gallery is part of exactly one regular exhibition (1 to 1..*).
Composition	«dataType» Feature (featureType, featureValue) is a multi-valued part of FeaturedExhibition (1..*) and LoyalVisitor (0..*).
Association classes (relationship attributes)	Borrowing and Lending (loans), ReplicaSale (on *includes*), Invitation (on *invited*), Maintains, SupervisesGallery, SupervisesFeatured.
Plain associations	ArtObject–Artist (*created by*, 0..1 : 0..*), ArtObject–Gallery (*displayed in*, 0..1 : 0..*), FeaturedExhibition–ArtObject (*includes*, 0..* : 1..*).

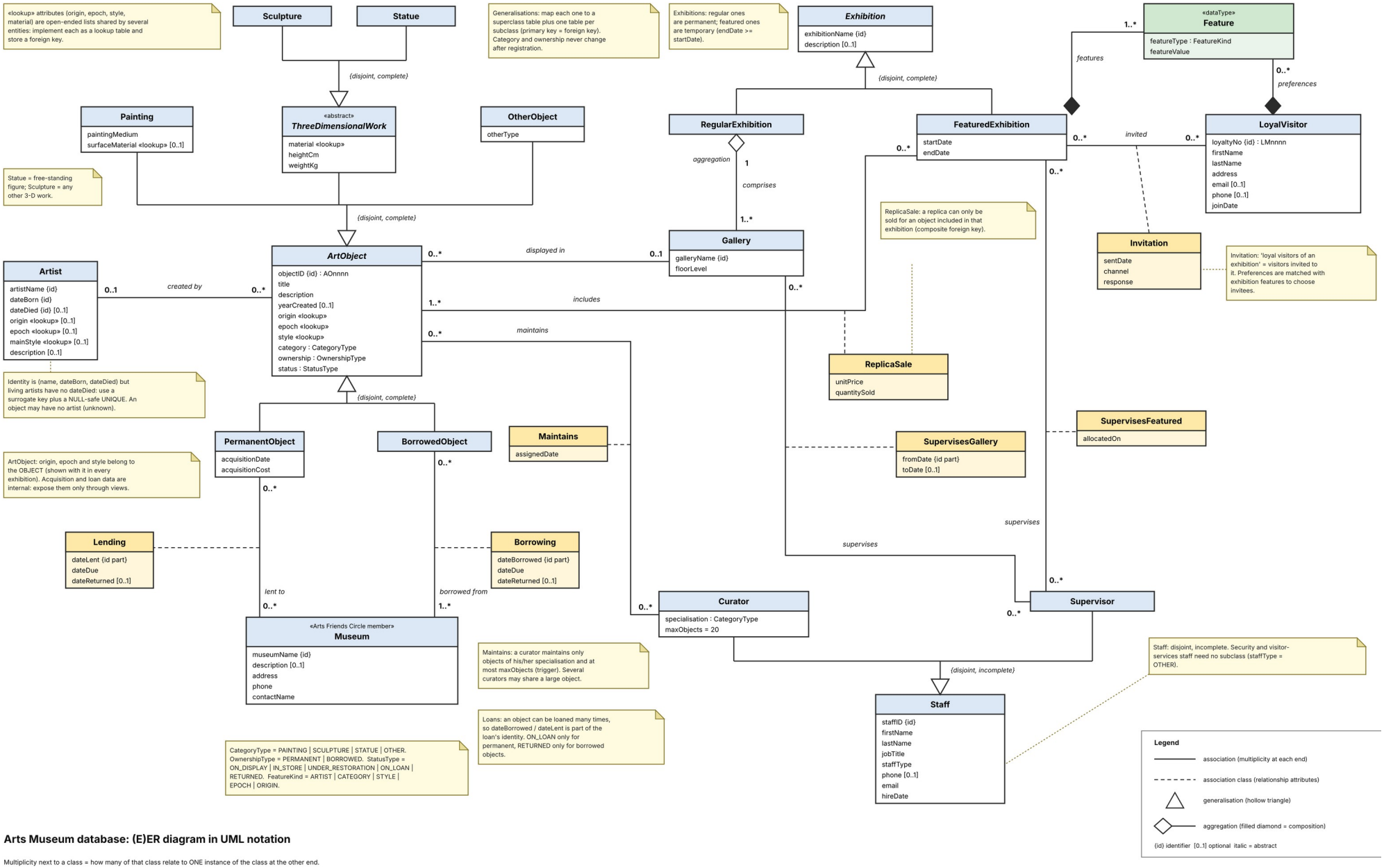


Figure 1 – (E)ER diagram of the Arts Museum database (full diagram)

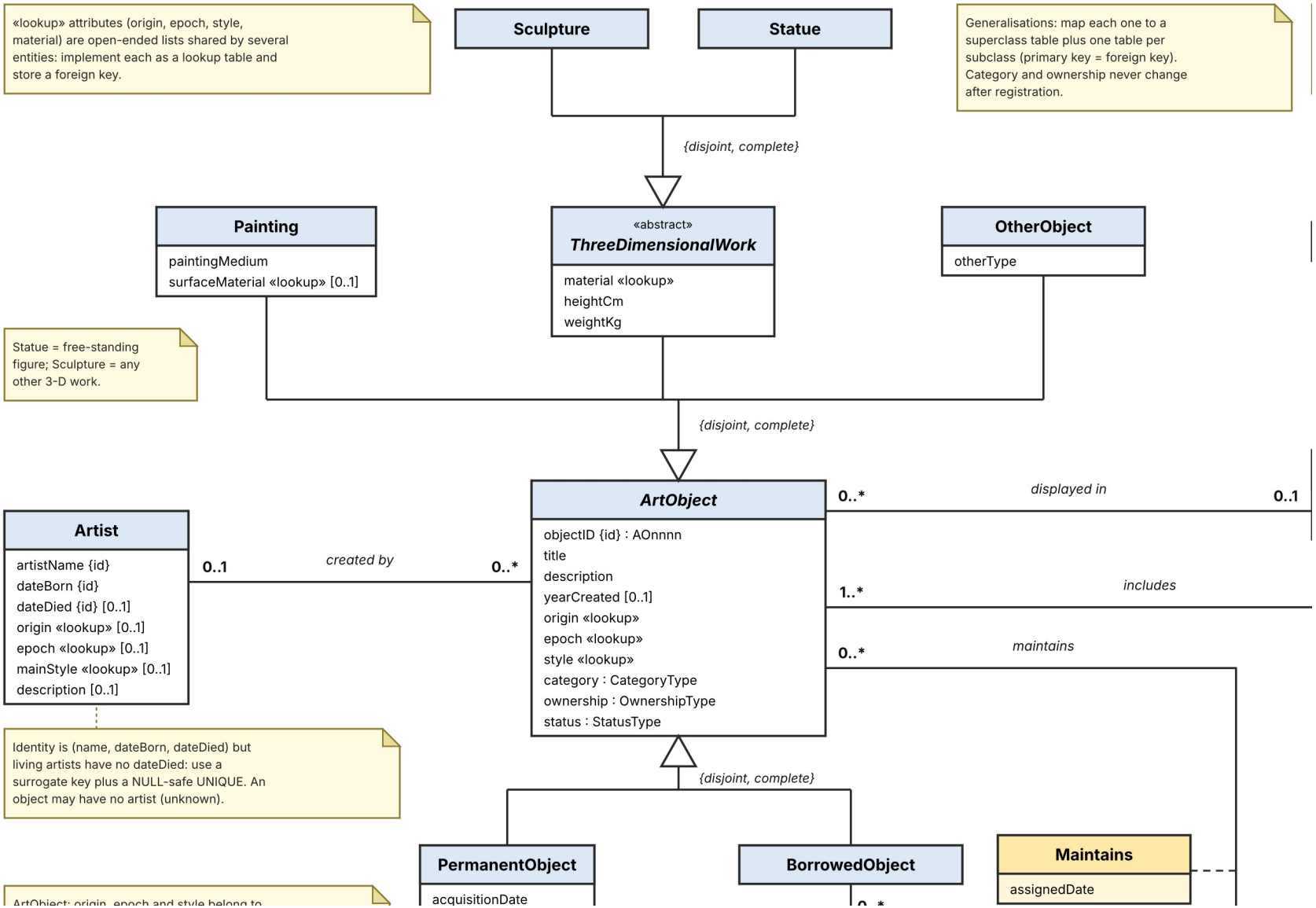
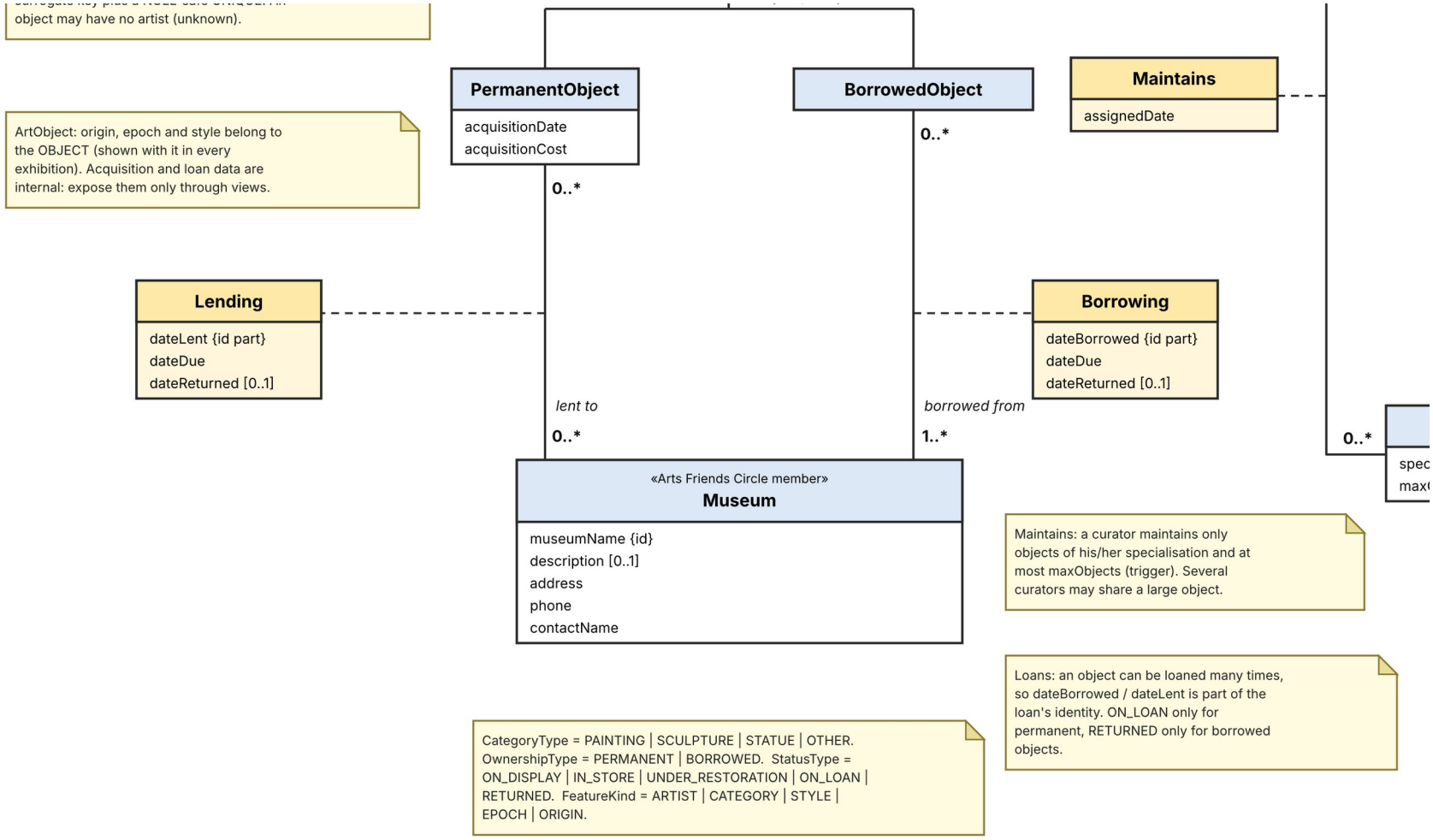


Figure 2 – Art objects, hierarchies and artists (enlargement)



Arts Museum database: (E)ER diagram in UML notation

Multiplicity next to a class = how many of that class relate to ONE instance of the class at the other end.

Figure 3 – Loans and the Arts Friends Circle (enlargement)

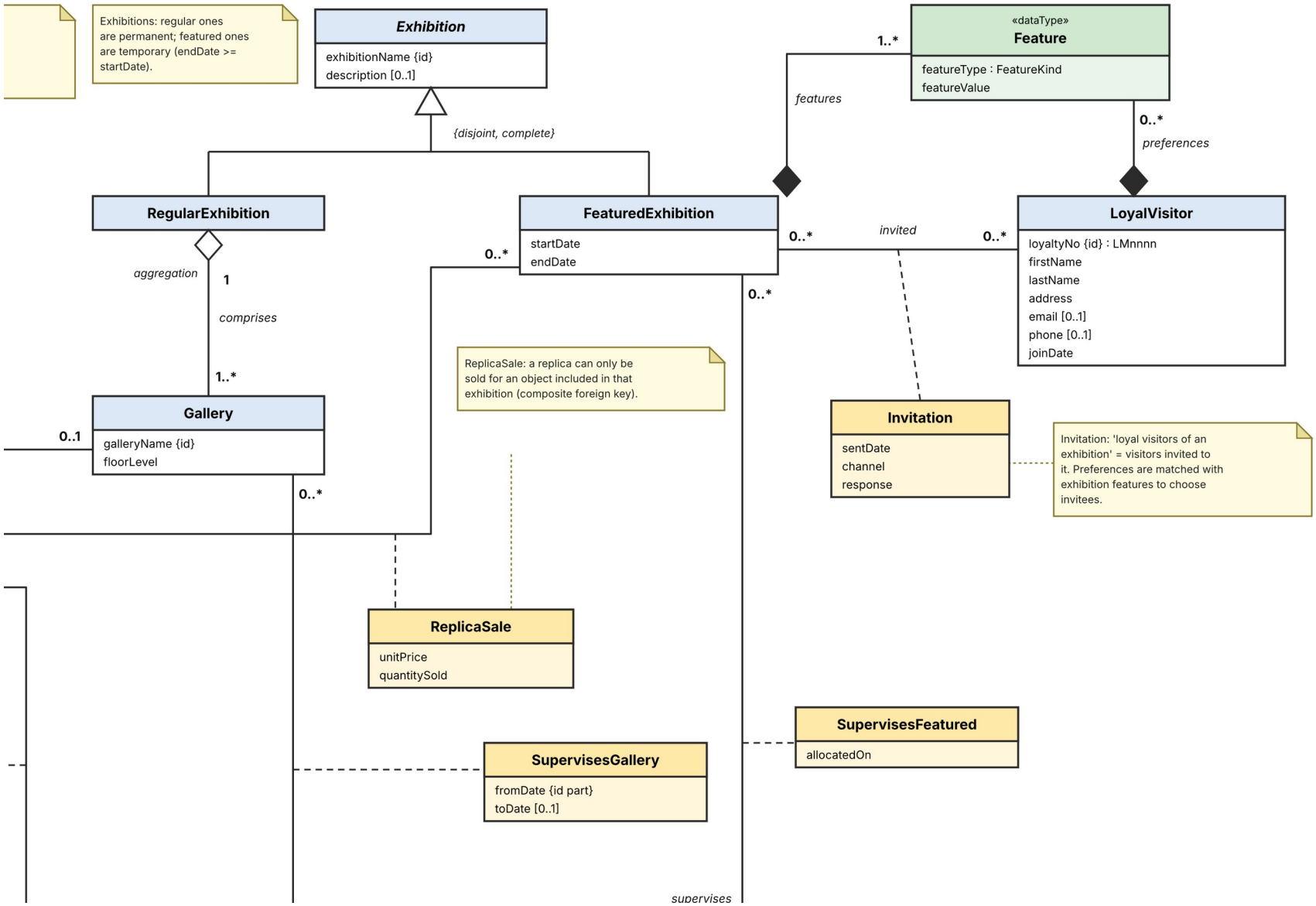


Figure 4 – Exhibitions, galleries, replicas and loyal visitors (enlargement)

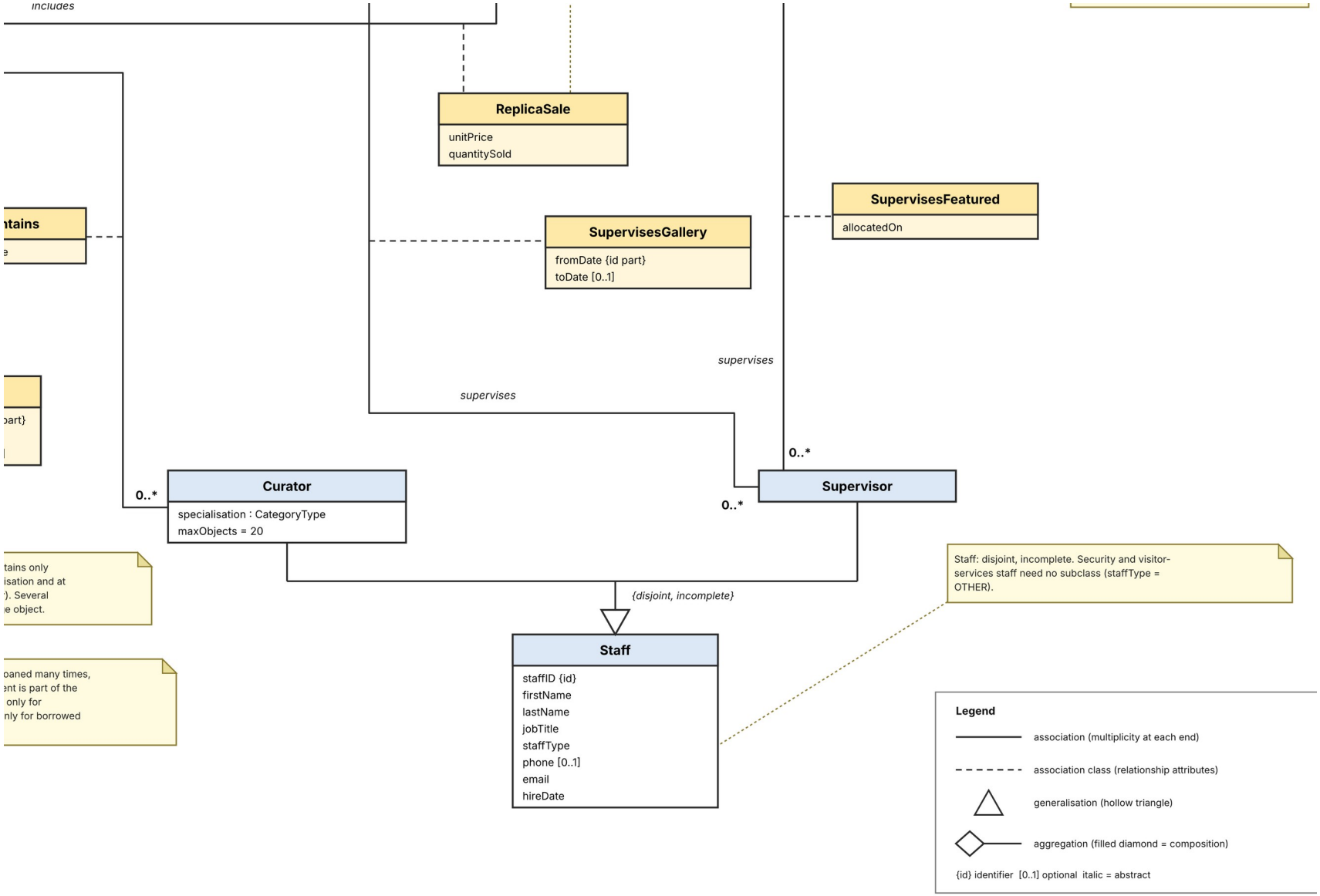


Figure 5 – Staff, curators, supervisors and their allocations (enlargement)

2.2 Assumptions

- **A1.** The Museum entity represents the *other* members of the Arts Friends Circle; this museum is not stored in it.
- **A2.** An art object has at most one artist; collaborations are not modelled (they would turn *created by* into a many-to-many relationship).
- **A3.** *Displayed in* records only the **current** gallery of an object, so an object is in at most one gallery at a time. Featured exhibitions keep a list of objects, so history is kept there.
- **A4.** An object can take part in many featured exhibitions, and a featured exhibition has at least one object.
- **A5.** The same object can be borrowed (or lent) many times, possibly from (to) different members; therefore the date of the loan is part of the identity of a loan.
- **A6.** Exhibition features and visitor preferences use one vocabulary (artist name, category, style, epoch, origin) and are matched by equality of type and value.
- **A7.** A curator has exactly one specialisation category and one maximum workload.
- **A8.** Supervisor allocations may overlap in time (one supervisor can cover two galleries).
- **A9.** A new invitation has the response *pending* until the visitor answers.
- **A10.** Replica sales are recorded as totals per exhibition and object, not as individual till receipts.
- **A11.** Money is in pounds sterling, heights in centimetres, weights in kilograms.
- **A12.** Dates are not validated against today's date, so future featured exhibitions can be planned in advance.
- **A13.** Staff of type *other* (security, visitor services) have no data beyond the common staff attributes.
- **A14.** Gallery names and exhibition names are unique.

2.3 Design decisions recorded as annotations

- **Superclass tables plus one table per subclass.** ArtObject takes part in almost every relationship (artist, gallery, loans, exhibitions, curators), so a relationship must point at one superclass table; the subclasses add only their own attributes. The annotation on the generalisation tells the relational designer to use a primary key that is also a foreign key.
- **Two independent generalisations.** Combining category and ownership into one hierarchy would need 4 × 2 = 8 subclasses and would copy every relationship eight times. Keeping them independent needs 6 subclasses.
- **Abstract ThreeDimensionalWork.** Material, height and weight are identical for sculptures and statues, so they are declared once.
- **Lookup attributes.** Origin, epoch, style and material are open lists ("etc.") shared by several entities; the note asks for lookup tables so each value has one spelling.
- **Artist identity.** The brief's identifier cannot work as a key for living artists; the note on Artist asks for a surrogate key plus a NULL-safe unique rule.
- **Display information.** Origin, epoch and style are shown with an object in every exhibition; acquisition and loan data are not. This is recorded as a note so the designer builds a view.
- **Loan history.** Because an object can be loaned repeatedly, Borrowing and Lending are identified by object **and** date.
- **Curator rules.** Specialisation and maximum workload involve two tables and cannot be declared with a simple constraint; the note asks for a trigger.

Task 3 – Relational mapping

3.1 How the EER model was mapped

We applied the mapping guidelines from the lab classes (Connolly and Begg, 2015) construct by construct. The complete schema is given, normalised, in Task 4; this table shows the decisions.

EER construct	Rule applied	Result
Strong entity	One relation per entity; attributes become columns; identifier becomes the primary key	Museum, Artist, Gallery, Exhibition, LoyalVisitor, Staff, ArtObject
Generalisation (all four)	Superclass relation plus one relation per subclass; the subclass key is the superclass key and also a foreign key; a discriminator column in the superclass	ArtObject + Painting, Sculpture, Statue, OtherObject, PermanentObject, BorrowedObject; Exhibition + RegularExhibition, FeaturedExhibition; Staff + Curator, Supervisor
Abstract intermediate class	Attributes pushed down into the concrete subclasses	material, heightCm, weightKg appear in Sculpture and Statue
One-to-many association	Foreign key on the "many" side; optional end gives a nullable column	ArtObject.artistID (nullable), ArtObject.galleryID (nullable)
Aggregation	Foreign key in the part	Gallery.regularExhibitionID
Many-to-many association	Junction relation with both keys as a compound primary key	ExhibitionObject, Maintains
Association class	Junction relation that also carries the relationship attributes; a date is added to the key where the same pair can repeat	Borrowing, Lending, ReplicaSale, Invitation, SupervisesGallery, SupervisesFeatured
Multi-valued composite attribute	Separate relation: owner's key plus every component as a compound key	ExhibitionFeature, VisitorPreference
«lookup» attribute	Lookup relation (id, name) and a foreign key	Origin, Epoch, Style, Material
Composite identifier with optional part	Surrogate primary key; the natural identifier kept as an alternate key	Artist (name, date born, date died)
Optional attribute	Nullable column	dateDied, yearCreated, email, …

3.2 The top challenge: mapping ArtObject's two generalisations

The problem. ArtObject is specialised in two independent ways (four categories, two kinds of ownership) and, at the same time, almost every other relationship of the database points at it: its artist, its gallery, its loans, the exhibitions that include it, the curators who maintain it. The relational model has no inheritance, so we had to choose how to spread one conceptual entity over several relations without losing integrity.

The options we weighed.

1. *One relation holding every attribute of every subclass.* Simple joins, but every row carries columns that never apply to it (a statue has no painting medium, a permanent object no lender), so NOT NULL cannot be declared for any subclass attribute and the relation mixes unrelated facts.
2. *Only subclass relations, no superclass relation.* This removes the NULLs, but a foreign key can reference only **one** relation. Exhibitions, curators and loans would each need a separate foreign key per subclass, or a union that SQL cannot reference. With two independent hierarchies we would also need 4 × 2 = 8 combined relations.
3. *A superclass relation plus one relation per subclass, sharing the primary key.* Every relationship can reference ArtObject, subclass attributes live in a relation of their own and can be NOT NULL, and the six subclasses (not eight combinations) are enough.

Our choice. Option 3, with two discriminator columns in ArtObject (`category` and `ownership`). Loans are attached to the *right* subclass: Borrowing references BorrowedObject and Lending references PermanentObject, so a permanent object cannot be recorded as borrowed.

What it costs, and how we paid. Plain foreign keys cannot (a) force a subclass row to match the discriminator, (b) force every object to have a subclass row, or (c) stop a discriminator changing afterwards. For (a) and (c) we use triggers (Task 8); (b) is guaranteed by writing an object in a single database transaction (transaction T1). Reading a complete painting needs one join, which we accept.

A second, smaller challenge was the identity of an artist: name, date born and date died cannot be a primary key because a living artist has no death date and a key cannot contain a missing value. We use a surrogate key, keep the three attributes as an alternate key, and make the uniqueness rule treat "no death date" as a value (Task 8).

3.3 Assumptions made while designing the schema

- Surrogate integer keys are used for Museum, Artist, Exhibition, Gallery, Staff and the four lookups, because their natural identifiers are long, can change, or are not defined by the brief. The brief's own identifiers (AOxxxx, LMxxxx) are kept as primary keys of ArtObject and LoyalVisitor. The museum name stays unique (alternate key).
- Closed lists (category, ownership, status, staff type, feature type, invitation channel and response) are domain constraints; open lists ("etc.") are lookup relations.
- An address is stored as one atomic value, because no requirement searches or sorts by town or postcode. Splitting it would also introduce a postcode-to-town dependency.
- Reference data is protected (`ON DELETE RESTRICT`); rows that exist only as part of another row are removed with it (`ON DELETE CASCADE`).
- A featured exhibition's features and a visitor's preferences are stored as (type, value) pairs and matched by equality; the values are not themselves foreign keys.

Task 4 – Normalised relational schemas (3NF)

4.1 Method

For every relation produced in Task 3 we (1) wrote down the functional dependencies that follow from the refined requirements, (2) checked **1NF**: all values atomic, no repeating groups, a primary key, (3) checked **2NF**: in 1NF and no non-key attribute depends on part of a compound key, (4) checked **3NF**: in 2NF and no non-key attribute depends on another non-key attribute. We also tested **BCNF** (every determinant is a candidate key); all 31 relations satisfy it, so the schema is in 3NF and in BCNF. The dependencies are listed in section 4.5 and were checked mechanically against the real column lists of the MySQL schema.

4.2 What the normalisation changed

Applying the mapping rules to the EER model gives a first set of relations. Normalisation found the following points, which are reflected in the final schemas of section 4.4.

- **N1 – Repeating groups (1NF).** The conceptual entities LoyalVisitor and FeaturedExhibition carry a *set* of features. A column holding several values is not atomic, so each set became its own relation: **VisitorPreference** (loyaltyNo, featureType, featureValue) and **ExhibitionFeature** (exhibitionID, featureType, featureValue).
- **N2 – Repeating strings (redundancy).** Origin, epoch, style and material names were repeated in many rows ("Italian" in every Italian object). Each became a lookup relation, so a name is stored once and an object holds its key.
- **N3 – Copied attributes (2NF and 3NF).** Whenever the requirements mention "the museum from which it was borrowed" or "the artist of the object", the tempting design copies the museum's or the artist's details into the loan or object relation. That creates partial and transitive dependencies. Worked example B shows how it is avoided.
- **N4 – Orthogonal subclass attributes.** Curator specialisation and maximum workload depend on the staff member alone, so they sit in Curator, not in Maintains (where they would depend on part of the key) and not in Staff (where they would be NULL for most rows).
- **N5 – The artist is not a determinant of the object's origin, epoch or style.** It would be a transitive dependency (objectID → artistID → originID) if the origin of an object were always the origin of its artist. It is not: the brief records origin and epoch for the object itself, they are displayed with it, they differ from the artist's (a Dutch painter working in France), and they exist for objects whose artist is unknown. So these columns stay in ArtObject and no dependency is hidden.
- **N6 – ReplicaSale.** The replica price depends on the exhibition **and** the object (the same replica may be priced differently at different exhibitions), so unitPrice depends on the whole key and the relation is in 2NF. Had the price depended on the object alone, it would have had to move to ArtObject.

Worked example A – invitations (from unnormalised form to 3NF)

Start from what the museum might keep on one sheet per visitor:

```
VisitorSheet (loyaltyNo, visitorName, visitorAddress, {preferenceType, preferenceValue},
{exhibitionID, exhibitionName, startDate, endDate, sentDate, channel, response})
```

The braces mark repeating groups, so the relation is **not even in 1NF**.

1NF – remove each repeating group into a relation of its own, carrying the owner's key:

- `LoyalVisitor (loyaltyNo, visitorName, visitorAddress)`
- `VisitorPreference (loyaltyNo, preferenceType, preferenceValue)`, key (loyaltyNo, preferenceType, preferenceValue)
- `VisitorExhibition (loyaltyNo, exhibitionID, exhibitionName, startDate, endDate, sentDate, channel, response)`, key (loyaltyNo, exhibitionID)

2NF – in VisitorExhibition, `exhibitionName`, `startDate` and `endDate` depend on `exhibitionID` alone, which is **part** of the key (partial dependency). They move out:

- `FeaturedExhibition (exhibitionID, exhibitionName, startDate, endDate)`

- `Invitation (loyaltyNo, exhibitionID, sentDate, channel, response)`, where every attribute depends on the whole key.

3NF – no non-key attribute determines another in any of the four relations, so they are in 3NF. (In the final schema the exhibition name moves one level further into the superclass relation Exhibition, because it is common to regular and featured exhibitions.)

Worked example B – loans (from first normal form to 3NF)

The first attempt at recording a loan puts everything in one relation:

`BorrowedObjectLoan (objectID, dateBorrowed, title, category, artistName, museumName, museumAddress, museumPhone, contactName, dateDue, dateReturned)`, key (objectID, dateBorrowed). All values are atomic, so it is in **1NF**.

2NF – `title`, `category` and `artistName` depend on `objectID` alone (partial dependencies); everything else depends on the full key (the object *and* the date). Move the first group to the object's own relation:

- `ArtObject (objectID, title, category, artistName)`
- `BorrowedObjectLoan (objectID, dateBorrowed, museumName, museumAddress, museumPhone, contactName, dateDue, dateReturned)`

3NF – in the loan relation `museumAddress`, `museumPhone` and `contactName` depend on `museumName`, a **non-key** attribute (transitive dependency); in ArtObject `artistName` depends on the artist, not on the object (the artist's details would repeat for every object). Remove both:

- `Museum (museumID, museumName, address, phone, contactName)` and `Borrowing (objectID, dateBorrowed, museumID, dateDue, dateReturned)`
- `Artist (artistID, artistName, …)` and `ArtObject (objectID, title, category, artistID, …)`

These are exactly the Museum, Borrowing, Artist and ArtObject relations of the final schema. The same reasoning gives Lending, which is analysed in Task 5.

4.3 Conventions used in the schemas

Each relation is written as in the lectures: `Relation (attributes)` followed by its keys. **Primary Key** is the primary key; **Alternate Key** is another candidate key; **Foreign Key** `a references R(b)`; **Domain** lists the allowed values of a column; **Check** is a row constraint; **Optional** lists the columns that may be NULL. Columns that are part of a primary key are never optional.

4.4 The normalised relational schemas

```
Origin (originID, originName)
    Primary Key    originID
    Alternate Key  originName

Epoch (epochID, epochName)
    Primary Key    epochID
    Alternate Key  epochName

Style (styleID, styleName)
    Primary Key    styleID
    Alternate Key  styleName

Material (materialID, materialName)
    Primary Key    materialID
    Alternate Key  materialName

Museum (museumID, museumName, description, address, phone, contactName)
    Primary Key    museumID
    Alternate Key  museumName
    Optional       description (NULL allowed)

Artist (artistID, artistName, dateBorn, dateDied, originID, epochID, mainStyleID, description)
    Primary Key    artistID
    Alternate Key  (artistName, dateBorn, dateDied)
    Foreign Key    originID references Origin(originID)
    Foreign Key    epochID references Epoch(epochID)
    Foreign Key    mainStyleID references Style(styleID)
    Check          dateDied IS NULL OR dateDied >= dateBorn
    Optional       dateDied, originID, epochID, mainStyleID, description (NULL allowed)
    Implementation dateDiedKey is a generated helper column (see Task 8)
```

```
Exhibition (exhibitionID, exhibitionName, description, exhibitionKind)
    Primary Key    exhibitionID
    Alternate Key  exhibitionName
    Domain         exhibitionKind in {REGULAR, FEATURED}
    Optional       description (NULL allowed)

RegularExhibition (exhibitionID)
    Primary Key    exhibitionID
    Foreign Key    exhibitionID references Exhibition(exhibitionID)

FeaturedExhibition (exhibitionID, startDate, endDate)
    Primary Key    exhibitionID
    Foreign Key    exhibitionID references Exhibition(exhibitionID)
    Check          endDate >= startDate

Gallery (galleryID, galleryName, floorLevel, regularExhibitionID)
    Primary Key    galleryID
    Alternate Key  galleryName
    Foreign Key    regularExhibitionID references RegularExhibition(exhibitionID)

ArtObject (objectID, title, description, yearCreated, category, ownership, status, artistID,
           originID, epochID, styleID, galleryID)
    Primary Key    objectID
    Foreign Key    artistID references Artist(artistID)
    Foreign Key    originID references Origin(originID)
    Foreign Key    epochID references Epoch(epochID)
    Foreign Key    styleID references Style(styleID)
    Foreign Key    galleryID references Gallery(galleryID)
    Domain         category in {PAINTING, SCULPTURE, STATUE, OTHER}
    Domain         ownership in {PERMANENT, BORROWED}
    Domain         status in {ON_DISPLAY, IN_STORE, UNDER_RESTORATION, ON_LOAN, RETURNED}
    Check          objectID REGEXP '^AO[0-9]{4}$'
    Check          yearCreated IS NULL OR yearCreated BETWEEN -5000 AND 2100
    Check          (ownership = 'PERMANENT' AND status <> 'RETURNED') OR (ownership = 'BORROWED' AND
                   status <> 'ON_LOAN')
    Optional       yearCreated, artistID, galleryID (NULL allowed)

Painting (objectID, paintingMedium, surfaceMaterialID)
    Primary Key    objectID
    Foreign Key    objectID references ArtObject(objectID)
    Foreign Key    surfaceMaterialID references Material(materialID)
    Optional       surfaceMaterialID (NULL allowed)

Sculpture (objectID, materialID, heightCm, weightKg)
    Primary Key    objectID
    Foreign Key    objectID references ArtObject(objectID)
    Foreign Key    materialID references Material(materialID)
    Check          heightCm > 0 AND weightKg > 0

Statue (objectID, materialID, heightCm, weightKg)
    Primary Key    objectID
    Foreign Key    objectID references ArtObject(objectID)
    Foreign Key    materialID references Material(materialID)
    Check          heightCm > 0 AND weightKg > 0

OtherObject (objectID, otherType)
    Primary Key    objectID
    Foreign Key    objectID references ArtObject(objectID)

PermanentObject (objectID, acquisitionDate, acquisitionCost)
    Primary Key    objectID
    Foreign Key    objectID references ArtObject(objectID)
    Check          acquisitionCost >= 0

BorrowedObject (objectID)
    Primary Key    objectID
    Foreign Key    objectID references ArtObject(objectID)

Borrowing (objectID, dateBorrowed, museumID, dateDue, dateReturned)
    Primary Key    (objectID, dateBorrowed)
    Foreign Key    objectID references BorrowedObject(objectID)
    Foreign Key    museumID references Museum(museumID)
    Check          dateDue >= dateBorrowed
    Check          dateReturned IS NULL OR dateReturned >= dateBorrowed
    Optional       dateReturned (NULL allowed)

Lending (objectID, dateLent, museumID, dateDue, dateReturned)
    Primary Key    (objectID, dateLent)
    Foreign Key    objectID references PermanentObject(objectID)
    Foreign Key    museumID references Museum(museumID)
    Check          dateDue >= dateLent
    Check          dateReturned IS NULL OR dateReturned >= dateLent
    Optional       dateReturned (NULL allowed)

ExhibitionFeature (exhibitionID, featureType, featureValue)
    Primary Key    (exhibitionID, featureType, featureValue)
    Foreign Key    exhibitionID references FeaturedExhibition(exhibitionID)
```

```
    Domain         featureType in {ARTIST, CATEGORY, STYLE, EPOCH, ORIGIN}

ExhibitionObject (exhibitionID, objectID)
    Primary Key    (exhibitionID, objectID)
    Foreign Key    exhibitionID references FeaturedExhibition(exhibitionID)
    Foreign Key    objectID references ArtObject(objectID)

ReplicaSale (exhibitionID, objectID, unitPrice, quantitySold)
    Primary Key    (exhibitionID, objectID)
    Foreign Key    (exhibitionID, objectID) references ExhibitionObject(exhibitionID, objectID)
    Check          unitPrice >= 0

LoyalVisitor (loyaltyNo, firstName, lastName, address, email, phone, joinDate)
    Primary Key    loyaltyNo
    Alternate Key  email
    Check          loyaltyNo REGEXP '^LM[0-9]{4}$'
    Optional       email, phone (NULL allowed)

VisitorPreference (loyaltyNo, featureType, featureValue)
    Primary Key    (loyaltyNo, featureType, featureValue)
    Foreign Key    loyaltyNo references LoyalVisitor(loyaltyNo)
    Domain         featureType in {ARTIST, CATEGORY, STYLE, EPOCH, ORIGIN}

Invitation (loyaltyNo, exhibitionID, sentDate, channel, response)
    Primary Key    (loyaltyNo, exhibitionID)
    Foreign Key    loyaltyNo references LoyalVisitor(loyaltyNo)
    Foreign Key    exhibitionID references FeaturedExhibition(exhibitionID)
    Domain         channel in {EMAIL, POST, SMS}
    Domain         response in {PENDING, ACCEPTED, DECLINED}

Staff (staffID, firstName, lastName, jobTitle, staffType, phone, email, hireDate)
    Primary Key    staffID
    Alternate Key  email
    Domain         staffType in {CURATOR, SUPERVISOR, OTHER}
    Optional       phone (NULL allowed)

Curator (staffID, specialisation, maxObjects)
    Primary Key    staffID
    Foreign Key    staffID references Staff(staffID)
    Domain         specialisation in {PAINTING, SCULPTURE, STATUE, OTHER}
    Check          maxObjects BETWEEN 1 AND 100

Supervisor (staffID)
    Primary Key    staffID
    Foreign Key    staffID references Staff(staffID)

Maintains (staffID, objectID, assignedDate)
    Primary Key    (staffID, objectID)
    Foreign Key    staffID references Curator(staffID)
    Foreign Key    objectID references ArtObject(objectID)

SupervisesGallery (staffID, galleryID, fromDate, toDate)
    Primary Key    (staffID, galleryID, fromDate)
    Foreign Key    staffID references Supervisor(staffID)
    Foreign Key    galleryID references Gallery(galleryID)
    Check          toDate IS NULL OR toDate >= fromDate
    Optional       toDate (NULL allowed)

SupervisesFeatured (staffID, exhibitionID, allocatedOn)
    Primary Key    (staffID, exhibitionID)
    Foreign Key    staffID references Supervisor(staffID)
    Foreign Key    exhibitionID references FeaturedExhibition(exhibitionID)
```

4.5 Functional dependencies and normal forms

For every relation, the table lists the non-trivial functional dependencies implied by the requirements and the highest normal form reached. In each relation every non-key attribute is determined by the whole primary key (2NF) and by nothing else (3NF); every determinant is a candidate key (BCNF).

Relation	Primary key	Non-trivial functional dependencies	Highest normal form
Origin	originID	originID -> originName; originName -> originID. Both determinants are candidate keys.	BCNF
Epoch	epochID	epochID -> epochName; epochName -> epochID. Both determinants are candidate keys.	BCNF
Style	styleID	styleID -> styleName; styleName -> styleID. Both determinants are candidate keys.	BCNF
Material	materialID	materialID -> materialName; materialName -> materialID. Both determinants are candidate keys.	BCNF

Relation	Primary key	Non-trivial functional dependencies	Highest normal form
Museum	museumID	museumID -> museumName, description, address, phone, contactName; museumName -> museumID. Address kept as one atomic value; contactName depends on the museum, not on phone.	BCNF
Artist	artistID	artistID -> artistName, dateBorn, dateDied, originID, epochID, mainStyleID, description; artistName, dateBorn, dateDied -> artistID. originID does not determine epochID or mainStyleID (artists of one culture span epochs and styles).	BCNF
Exhibition	exhibitionID	exhibitionID -> exhibitionName, description, exhibitionKind; exhibitionName -> exhibitionID. Dates are not here: they depend only on featured exhibitions.	BCNF
RegularExhibition	exhibitionID	none beyond the key. Key-only relation (subclass membership).	BCNF
FeaturedExhibition	exhibitionID	exhibitionID -> startDate, endDate. Only one determinant, the key.	BCNF
Gallery	galleryID	galleryID -> galleryName, floorLevel, regularExhibitionID; galleryName -> galleryID. regularExhibitionID does not determine floorLevel.	BCNF
ArtObject	objectID	objectID -> title, description, yearCreated, category, ownership, status, artistID, originID, epochID, styleID, galleryID. artistID does NOT determine originID/epochID/styleID: the origin, epoch and style of an object are recorded per object, may differ from the artist's, and exist for unknown artists.	BCNF
Painting	objectID	objectID -> paintingMedium, surfaceMaterialID. surfaceMaterialID is a foreign key, not a copied name.	BCNF
Sculpture	objectID	objectID -> materialID, heightCm, weightKg. Dimensions depend only on the object.	BCNF
Statue	objectID	objectID -> materialID, heightCm, weightKg. Dimensions depend only on the object.	BCNF
OtherObject	objectID	objectID -> otherType. Only one determinant, the key.	BCNF
PermanentObject	objectID	objectID -> acquisitionDate, acquisitionCost. Only one determinant, the key.	BCNF
BorrowedObject	objectID	none beyond the key. Key-only relation (subclass membership).	BCNF
Borrowing	objectID, dateBorrowed	objectID, dateBorrowed -> museumID, dateDue, dateReturned. Lender details live in Museum (copying them here would create museumID -> museum details).	BCNF
Lending	objectID, dateLent	objectID, dateLent -> museumID, dateDue, dateReturned. Borrower details live in Museum; object details live in ArtObject.	BCNF
ExhibitionFeature	exhibitionID, featureType, featureValue	exhibitionID, featureType, featureValue -> exhibitionID. All-key relation.	BCNF
ExhibitionObject	exhibitionID, objectID	exhibitionID, objectID -> exhibitionID. All-key relation.	BCNF
ReplicaSale	exhibitionID, objectID	exhibitionID, objectID -> unitPrice, quantitySold. The price may differ per exhibition, so unitPrice depends on the whole key, not on objectID alone.	BCNF
LoyalVisitor	loyaltyNo	loyaltyNo -> firstName, lastName, address, email, phone, joinDate; email -> loyaltyNo (when not NULL). email is an alternate key.	BCNF
VisitorPreference	loyaltyNo, featureType, featureValue	loyaltyNo, featureType, featureValue -> loyaltyNo. All-key relation.	BCNF
Invitation	loyaltyNo, exhibitionID	loyaltyNo, exhibitionID -> sentDate, channel, response. Visitor and exhibition details live in their own relations.	BCNF
Staff	staffID	staffID -> firstName, lastName, jobTitle, staffType, phone, email, hireDate; email -> staffID. jobTitle does not determine staffType (several titles per type).	BCNF
Curator	staffID	staffID -> specialisation, maxObjects. specialisation does not determine maxObjects: workload is an individual agreement.	BCNF
Supervisor	staffID	none beyond the key. Key-only relation (subclass membership).	BCNF
Maintains	staffID, objectID	staffID, objectID -> assignedDate. specialisation/maxObjects stay in Curator (they would depend on staffID alone).	BCNF
SupervisesGallery	staffID, galleryID, fromDate	staffID, galleryID, fromDate -> toDate. toDate depends on the whole key.	BCNF
SupervisesFeatured	staffID, exhibitionID	staffID, exhibitionID -> allocatedOn. allocatedOn depends on the whole key.	BCNF

Task 5 – Why Lending is in third normal form

We chose **Lending**, a relation with a compound primary key:

```
Lending (objectID, dateLent, museumID, dateDue, dateReturned)
    Primary Key    (objectID, dateLent)
    Foreign Key    objectID references PermanentObject(objectID)
    Foreign Key    museumID references Museum(museumID)
```

The functional dependency is **(objectID, dateLent) → museumID, dateDue, dateReturned**. A few rows of the sample data:

objectID	dateLent	museumID	dateDue	dateReturned
AO0003	2024-04-01	2	2024-10-01	2024-09-28
AO0007	2025-01-10	3	2025-07-10	2025-07-09
AO0019	2025-09-01	7	2026-02-28	2026-02-25
AO0024	2026-10-01	7	2027-01-31	NULL

First normal form. Every column holds a single atomic value (a date, a number); there are no repeating groups or lists; every row is identified by the primary key (objectID, dateLent), so there are no duplicate rows. "Not returned yet" is stored as one NULL in `dateReturned`, not as a list or a special code. The relation is in 1NF.

Second normal form. The relation is in 1NF and has a compound key, so we must show that no non-key attribute depends on only *part* of the key.

- *Does `museumID` depend on `objectID` alone?* No. The same object can be lent to different museums at different times. *On `dateLent` alone?* No. On 2025-09-01 different objects could be lent to different museums.
- *Does `dateDue` depend on `objectID` alone or on `dateLent` alone?* No, for the same reasons: each loan has its own due date.
- *Does `dateReturned`?* No: it is a fact about one loan, that is, about the pair (object, date lent).

Every non-key attribute therefore depends on the **whole** key and the relation is in 2NF. For contrast, a column `title` would depend on `objectID` alone (a partial dependency) and would break 2NF, which is why the title is stored only in ArtObject.

Third normal form. The relation is in 2NF and we must show that no non-key attribute depends on another non-key attribute (no transitive dependency). The non-key attributes are `museumID`, `dateDue` and `dateReturned`.

- `museumID` does not determine `dateDue` or `dateReturned`: the same museum borrows different objects for different periods.
- `dateDue` does not determine `dateReturned` (an object may come back early or late), and the reverse does not hold either.
- The only facts that *would* create a transitive dependency are the museum's name, address, phone or contact: they depend on `museumID`, a non-key attribute. They are deliberately **not** stored in Lending; they live in Museum, which is why Lending holds just the foreign key.

The only determinant in Lending is its primary key, so the relation is in 3NF and also in Boyce-Codd normal form. The relation Borrowing has the same structure.

Task 6 – A class/subclass relationship: ArtObject and Painting

The EER construct. In the EER diagram ArtObject is the superclass of Painting in the generalisation by category ({disjoint, complete}, with Painting, ThreeDimensionalWork and OtherObject as subclasses). Painting adds two attributes of its own: the painting medium and, optionally, the surface material.

How the relational schema models it.

```
ArtObject (objectID, title, description, yearCreated, category, ownership, status,
           artistID, originID, epochID, styleID, galleryID)
    Primary Key  objectID

Painting (objectID, paintingMedium, surfaceMaterialID)
    Primary Key  objectID
    Foreign Key  objectID references ArtObject(objectID)  ON DELETE CASCADE
```

- **Shared key.** `Painting.objectID` is both the primary key and a foreign key to `ArtObject.objectID`. A painting is *one* real-world thing described in two rows, so there is at most one Painting row per object (the primary key) and none without its ArtObject row (the foreign key). Deleting the ArtObject deletes the Painting row with it.
- **Discriminator.** `ArtObject.category` holds PAINTING, SCULPTURE, STATUE or OTHER. This is what makes the generalisation **disjoint**: an object has a single category value, so it can match only one subclass. A trigger (`trg_painting_bi`) rejects a Painting row whose ArtObject has any other category, and `trg_artobject_bu` forbids changing the category afterwards.
- **Attributes.** Everything common to all objects (title, year, origin, epoch, style, artist, gallery, status) is stored once in ArtObject; only painting-specific columns are in Painting, and they can be declared NOT NULL (`paintingMedium`) without affecting other categories.
- **Relationships.** Exhibitions, curators, loans and galleries reference ArtObject, so they work for every category without repeating a foreign key per subclass.

Using it.

```
-- the complete description of a painting: join the two rows
SELECT ao.objectID, ao.title, p.paintingMedium, m.materialName AS surface
FROM ArtObject ao
JOIN Painting p      ON p.objectID = ao.objectID
LEFT JOIN Material m ON m.materialID = p.surfaceMaterialID
WHERE ao.objectID = 'AO0001';
```

What cannot be declared. The *complete* part of {disjoint, complete} (every object with category PAINTING must have a Painting row) cannot be written as a foreign key, because the dependency points from the parent to the child. We guarantee it procedurally: a new object is always inserted in one transaction that writes the ArtObject row, its category row and its ownership row together (transaction T1), and the test script checks that every object has exactly one subclass row of each hierarchy.

Alternatives we rejected. A single table with all columns would leave sculpture and print columns empty in every painting row and could not declare any subclass attribute NOT NULL. A Painting table without an ArtObject table would force every other relationship to be duplicated for each category.

Task 7 – How the database fulfils your requirements

This section is written for the museum owner. Technical names appear in brackets so that your IT staff can find them.

What the database does for you

The database keeps one organised record of everything described in your brief: every art object, whether the museum owns it or has borrowed it; the artists; the other members of the Arts Friends Circle and every loan between you and them; the regular and featured exhibitions and the galleries; your loyal visitors, their preferences and the invitations you send them; and your staff, including who looks after which objects and which galleries. Because each fact is stored in exactly one place, a change (a new contact at another museum, a renamed gallery) is made once and appears everywhere. The rules of the museum are built into the database, so wrong entries are refused instead of being discovered later.

Your requirements, one by one

What you asked for	How the database delivers it
A large permanent collection, plus objects borrowed from other museums	Every object has one record and is marked as **permanent** or **borrowed**, never both. Permanent objects carry the date acquired and the cost; borrowed objects carry their loans.
Other museums can borrow from you	Each loan out of the museum is recorded (who, when, due back, returned). The database shows at once which of your objects are away and when they are due home (job 11).
The Arts Friends Circle, with name, description, address, phone and current contact	One record per member museum. Names cannot be repeated, and when a contact changes you change it once.
Unique number AOXXXX for each object	Every object has its own number AO0001, AO0002 …; the database refuses duplicates and badly formed numbers. Please note that four digits allow 10,000 objects; the pattern should be widened before you get near that.
Artist, year, title, description, origin and epoch for each object	All are stored with the object. Origin, epoch, style and materials are chosen from fixed lists, so "Italian" is never spelt three different ways. If the artist or the year is not known, you can leave it empty.
Paintings, sculptures, statues and "others", each with its own details	Each object is in exactly one of the four groups. A painting records its type (oil, watercolour …) and surface; a sculpture or statue its material, height and weight; an "other" object its type (print, photograph …). Style is recorded for all.
What visitors see, but not the internal details	A ready-made **view** shows only the public information: title, artist, year, description, origin and epoch (plus style, category and gallery). Costs, acquisition dates and loan details are not part of it.
Information about artists, identified by name, birth date and death date	One record per artist. Living artists simply have no death date, and the database still prevents the same artist being entered twice.
Regular and featured exhibitions; featured ones for specific periods	Regular exhibitions are permanent and are made of galleries; featured exhibitions have a start and an end date, and the database refuses an end date before the start (you can extend an exhibition, job 2).
Replicas sold during featured exhibitions	For each featured exhibition and object we record the replica price and how many were sold, so income per exhibition is one question (job 12). A replica can only be recorded for an object that is in that exhibition.
Invitations to loyal visitors with loyalty numbers LMYYYY	Each loyal visitor has a number (LM1001 …), contact details and preferences. Every invitation is recorded with date, channel and answer, and the database can suggest whom to invite next (job 13).
Several kinds of staff	Everyone has a staff record; curators and supervisors have the extra information they need. Other staff (security, visitor services) are recorded too.
Curators look after a certain number and type of objects; several curators for large objects	Each curator has a specialty and a maximum (20 unless agreed otherwise). The database **refuses** an assignment of the wrong type or beyond the maximum. An object can have several curators.
Supervisors allocated to galleries or to featured exhibitions	Allocations to galleries (with dates) and to featured exhibitions are recorded separately, so you can see who supervises what, now and in the past.

The jobs you can ask the database to do

1. Add a new object to the permanent collection (all its records are saved together, or not at all).
2. Change the end date of a featured exhibition.
3. List all borrowed objects, with the lender, dates and whether each has gone back.

4. Count how many objects a given artist created.
5. Count the objects on display in the regular exhibition per group, largest group first.
6. List all objects borrowed from a given member museum.
7. List the staff supervising a given featured exhibition.
8. List the curators responsible for a given object.
9. List the featured exhibitions with their dates and the loyal visitors invited.
10. List the objects of a featured exhibition made by a given artist.
11. List your objects that are on loan to other museums and not yet back.
12. Show replica income per featured exhibition.
13. List the loyal visitors whose tastes match an exhibition and who have not yet been invited.

Appendix A shows each job as a query together with the result obtained from the sample data.

What you should know

- **Decisions we made on your behalf** are listed in Task 1 (for example: borrowed objects also get a status; an object is "on display" exactly when it hangs in a gallery; "their loyal visitors" means the visitors who were invited).
- **Things staff must still do:** keep the status of an object up to date, record a loan when an object leaves or arrives, and register a new object completely.
- **Limits:** the database records invitations but does not send them; replica sales are recorded as totals per exhibition; exhibition features and visitor preferences are matched by name, so they must be spelt as in the lists.

Task 8 – MySQL implementation

The database is created by `01_schema.sql` (31 tables), the views and triggers by `03_views_and_triggers.sql`. All tables use the InnoDB engine (the only MySQL engine that enforces foreign keys and transactions) and the character set `utf8mb4`, so names such as "Élodie" are stored correctly. CHECK constraints are enforced from MySQL 8.0.16; in earlier versions they are accepted but silently ignored.

8.1 Choice of data types

Data type	Used for	Why
`CHAR(6)`	`objectID` (AO0001), `loyaltyNo` (LM1001)	The identifiers have a fixed length, so a fixed-length type is exact and compares quickly; a CHECK enforces the pattern.
`INT UNSIGNED AUTO_INCREMENT`	Surrogate keys of Museum, Artist, Exhibition, Gallery, Staff	Compact, never negative, generated by the database so no two rows can clash; 4 billion values are far more than needed.
`SMALLINT UNSIGNED AUTO_INCREMENT`	Keys of the lookup tables	Lookup lists are short; two bytes per foreign key saves space in the big ArtObject table.
`VARCHAR(n)`	Names, titles, addresses, phone numbers, emails	Variable length with a limit sized for the real data (60 for list names, 100 for person and museum names, 150 for titles, 200 for addresses, 25 for phone numbers).
`TEXT`	Descriptions	Length is open-ended; TEXT is not used for anything that is searched or joined.
`DATE`	Dates acquired, borrowed, lent, due, returned, born, died, sent, hired	No time of day is needed. A `DATE` compares correctly and supports date arithmetic; strings would not.
`SMALLINT` (signed)	`yearCreated`	Holds -32768 to 32767; negative years represent BCE objects, and a CHECK limits it to -5000 … 2100.
`TINYINT`	`floorLevel`; `TINYINT UNSIGNED` for `maxObjects`	One byte is enough for a floor number or a workload of at most 100.
`DECIMAL(12,2)`, `DECIMAL(8,2)`	Acquisition cost, replica price	Money must never be stored as `FLOAT`, which rounds; DECIMAL is exact to the penny.
`DECIMAL(7,2)`, `DECIMAL(9,2)`	Height in cm, weight in kg	Exact values with two decimals; ranges cover small prints up to multi-tonne statues.
`ENUM(...)`	Closed lists: category, ownership, status, staff type, exhibition kind, feature type, channel, response, specialisation	The database rejects any value that is not in the list, and each value takes one byte. Open lists use lookup tables instead.
`INT UNSIGNED` (counts)	`quantitySold`	Whole numbers, never negative.

8.2 Features that preserve data integrity

Feature	Where it is used	What it protects
PRIMARY KEY	Every table; compound keys in all junction tables	Every row is identifiable and none is duplicated.
FOREIGN KEY (39)	Every relationship	No loan without a museum, no painting without an object, no replica for an object that is not in the exhibition (composite key to ExhibitionObject).
UNIQUE	Museum name, exhibition and gallery names, staff and visitor email, lookup names	The "unique" rules of the brief and clean lookup lists.
Generated column + UNIQUE	`Artist.dateDiedKey = IFNULL(dateDied,'9999-12-31')`	Two living artists with the same name and birth date are rejected. A plain UNIQUE on (name, born, died) would accept both because MySQL treats NULLs as different.
NOT NULL	All mandatory columns	The brief's mandatory facts (title, description, origin, epoch, style, dates of a loan …) can never be missing. Optional facts (artist, year, death date, surface material) are the only NULLable columns.
DEFAULT	`status = 'IN_STORE'`, `maxObjects = 20`, `response = 'PENDING'`, `quantitySold = 0`, `floorLevel = 0`	Sensible starting values; 20 comes from the brief.

Feature	Where it is used	What it protects
CHECK (16)	Identifier patterns (`AO` + 4 digits, `LM` + 4 digits); `endDate >= startDate`; `dateDue >= dateBorrowed`; `dateReturned >= dateBorrowed`; `dateDied >= dateBorn`; cost and price not negative, height and weight positive; `maxObjects` between 1 and 100; year between -5000 and 2100; a permanent object cannot be RETURNED and a borrowed object cannot be ON_LOAN	Impossible values and impossible combinations of columns in one row.
ENUM	See 8.1	Closed lists.
Triggers (14)	Subclass rows must match the discriminator; ON_DISPLAY if and only if a gallery is set; category and ownership cannot change; a curator may only maintain objects of his/her specialisation and at most `maxObjects`	Rules that involve a second table, which CHECK and FOREIGN KEY cannot express.
Views (3)	`vw_ObjectDisplayInfo`, `vw_FeaturedExhibitionCatalogue`, `vw_BorrowedObjectsHeld`	Visitors see title, artist, year, description, origin and epoch (plus style, category and gallery); acquisition cost and loan details stay internal.

8.3 Referential actions

Foreign key points at	Action	Reason
Reference data: Artist, Museum, lookup tables, Gallery (from ArtObject), RegularExhibition (from Gallery)	`ON DELETE RESTRICT`	History must not vanish: an artist, museum, gallery or list value that is still in use cannot be deleted.
The superclass row (Painting → ArtObject, Curator → Staff, …)	`ON DELETE CASCADE`	A subclass row has no meaning without its superclass row.
Dependent rows (invitations, features, preferences, exhibition objects, replica sales, allocations, maintenance)	`ON DELETE CASCADE` from their owner	They exist only as part of the owner.
Loans (Borrowing, Lending) → objects	`ON DELETE RESTRICT`	An object with loan history cannot be deleted by accident.
Every foreign key	`ON UPDATE CASCADE`	If a key ever changes, children follow.

Gallery uses `RESTRICT` rather than `SET NULL` on purpose: MySQL does not fire triggers for actions cascaded by a foreign key, so `SET NULL` could leave an object ON_DISPLAY without a gallery.

8.4 CREATE TABLE statements

The full script `01_schema.sql` follows. It can be run as it is in MySQL Workbench or with `mysql < 01_schema.sql`.

```
-- ========================================================================
--  CS2019 Assessment 2 : Arts Museum Database
--  Script 1 of 4 : database and tables            Target : MySQL 8.0.16+ (InnoDB)
--  Conventions    : tables PascalCase, columns camelCase,
--                   constraint prefixes ck_ (CHECK), fk_ (FOREIGN KEY), uq_ (UNIQUE)
-- ========================================================================
DROP DATABASE IF EXISTS ArtsMuseumDB;
CREATE DATABASE ArtsMuseumDB CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE ArtsMuseumDB;

-- ------------------------------------------------------------------------
-- A. Controlled vocabularies. The brief lists open-ended values ("etc.") that
--    are shared by several entities, so each gets a lookup table: one spelling
--    per value and no repeated text.
-- ------------------------------------------------------------------------
CREATE TABLE Origin (
  originID   SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  originName VARCHAR(60)       NOT NULL,            -- country or culture, e.g. Italian
  PRIMARY KEY (originID),
  UNIQUE KEY uq_origin_name (originName)
) ENGINE=InnoDB;

CREATE TABLE Epoch (
  epochID   SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  epochName VARCHAR(60)       NOT NULL,            -- e.g. Renaissance, Modern, Ancient
  PRIMARY KEY (epochID),
  UNIQUE KEY uq_epoch_name (epochName)
) ENGINE=InnoDB;
```

```
CREATE TABLE Style (
  styleID   SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  styleName VARCHAR(60)       NOT NULL,             -- e.g. Classic, Modern, Abstract
  PRIMARY KEY (styleID),
  UNIQUE KEY uq_style_name (styleName)
) ENGINE=InnoDB;

CREATE TABLE Material (
  materialID   SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  materialName VARCHAR(60)       NOT NULL,          -- e.g. Canvas, Paper, Wood, Marble
  PRIMARY KEY (materialID),
  UNIQUE KEY uq_material_name (materialName)
) ENGINE=InnoDB;

-- ------------------------------------------------------------------------
-- B. Arts Friends Circle members and artists
-- ------------------------------------------------------------------------
CREATE TABLE Museum (
  museumID    INT UNSIGNED NOT NULL AUTO_INCREMENT,  -- surrogate key
  museumName  VARCHAR(100) NOT NULL,                 -- unique in the brief
  description TEXT         NULL,
  address     VARCHAR(200) NOT NULL,                 -- treated as one atomic value
  phone       VARCHAR(25)  NOT NULL,
  contactName VARCHAR(100) NOT NULL,                 -- current contact person
  PRIMARY KEY (museumID),
  UNIQUE KEY uq_museum_name (museumName)
) ENGINE=InnoDB;

CREATE TABLE Artist (
  artistID    INT UNSIGNED NOT NULL AUTO_INCREMENT,  -- surrogate key
  artistName  VARCHAR(100) NOT NULL,
  dateBorn    DATE         NOT NULL,
  dateDied    DATE         NULL,                     -- NULL = still living
  -- UNIQUE ignores NULLs, so living artists need a stand-in value to keep
  -- (name, dateBorn, dateDied) unique exactly as the brief demands.
  dateDiedKey DATE GENERATED ALWAYS AS (IFNULL(dateDied, '9999-12-31')) STORED,
  originID    SMALLINT UNSIGNED NULL,
  epochID     SMALLINT UNSIGNED NULL,
  mainStyleID SMALLINT UNSIGNED NULL,
  description TEXT         NULL,
  PRIMARY KEY (artistID),
  UNIQUE KEY uq_artist_identity (artistName, dateBorn, dateDiedKey),
  CONSTRAINT ck_artist_dates CHECK (dateDied IS NULL OR dateDied >= dateBorn),
  CONSTRAINT fk_artist_origin FOREIGN KEY (originID)
    REFERENCES Origin (originID) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_artist_epoch FOREIGN KEY (epochID)
    REFERENCES Epoch (epochID) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_artist_style FOREIGN KEY (mainStyleID)
    REFERENCES Style (styleID) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- ------------------------------------------------------------------------
-- C. Exhibitions (generalisation: Exhibition -> Regular | Featured) and galleries
-- ------------------------------------------------------------------------
CREATE TABLE Exhibition (
  exhibitionID   INT UNSIGNED NOT NULL AUTO_INCREMENT,
  exhibitionName VARCHAR(120) NOT NULL,
  description    TEXT         NULL,
  exhibitionKind ENUM('REGULAR','FEATURED') NOT NULL,   -- discriminator
  PRIMARY KEY (exhibitionID),
  UNIQUE KEY uq_exhibition_name (exhibitionName)
) ENGINE=InnoDB;

CREATE TABLE RegularExhibition (
  exhibitionID INT UNSIGNED NOT NULL,
  PRIMARY KEY (exhibitionID),
  CONSTRAINT fk_regular_exhibition FOREIGN KEY (exhibitionID)
    REFERENCES Exhibition (exhibitionID) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE FeaturedExhibition (
  exhibitionID INT UNSIGNED NOT NULL,
  startDate    DATE NOT NULL,
  endDate      DATE NOT NULL,
  PRIMARY KEY (exhibitionID),
  CONSTRAINT ck_featured_dates CHECK (endDate >= startDate),
  CONSTRAINT fk_featured_exhibition FOREIGN KEY (exhibitionID)
    REFERENCES Exhibition (exhibitionID) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE Gallery (                              -- part of one regular exhibition
  galleryID           INT UNSIGNED NOT NULL AUTO_INCREMENT,
  galleryName         VARCHAR(80)  NOT NULL,
  floorLevel          TINYINT      NOT NULL DEFAULT 0,  -- 0 = ground floor
  regularExhibitionID INT UNSIGNED NOT NULL,
  PRIMARY KEY (galleryID),
  UNIQUE KEY uq_gallery_name (galleryName),
```

```
  CONSTRAINT fk_gallery_regular FOREIGN KEY (regularExhibitionID)
    REFERENCES RegularExhibition (exhibitionID) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- ------------------------------------------------------------------------------
-- D. Art objects: superclass plus one table per subclass (shared primary key)
--    Two independent specialisations: category (4 subclasses) and ownership (2).
-- ------------------------------------------------------------------------------
CREATE TABLE ArtObject (
  objectID    CHAR(6)      NOT NULL,                  -- AOXXXX
  title       VARCHAR(150) NOT NULL,
  description TEXT         NOT NULL,
  yearCreated SMALLINT     NULL,                      -- negative = BCE, NULL = unknown
  category    ENUM('PAINTING','SCULPTURE','STATUE','OTHER') NOT NULL,   -- discriminator 1
  ownership   ENUM('PERMANENT','BORROWED') NOT NULL,                    -- discriminator 2
  status      ENUM('ON_DISPLAY','IN_STORE','UNDER_RESTORATION','ON_LOAN','RETURNED')
              NOT NULL DEFAULT 'IN_STORE',
  artistID    INT UNSIGNED NULL,                     -- NULL = unknown / anonymous artist
  originID    SMALLINT UNSIGNED NOT NULL,
  epochID     SMALLINT UNSIGNED NOT NULL,
  styleID     SMALLINT UNSIGNED NOT NULL,
  galleryID   INT UNSIGNED NULL,                     -- set only while ON_DISPLAY
  PRIMARY KEY (objectID),
  CONSTRAINT ck_ao_id CHECK (objectID REGEXP '^AO[0-9]{4}$'),
  CONSTRAINT ck_ao_year CHECK (yearCreated IS NULL OR yearCreated BETWEEN -5000 AND 2100),
  CONSTRAINT ck_ao_ownership_status CHECK (
        (ownership = 'PERMANENT' AND status <> 'RETURNED')
     OR (ownership = 'BORROWED'  AND status <> 'ON_LOAN')),
  CONSTRAINT fk_ao_artist FOREIGN KEY (artistID)
    REFERENCES Artist (artistID) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_ao_origin FOREIGN KEY (originID)
    REFERENCES Origin (originID) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_ao_epoch FOREIGN KEY (epochID)
    REFERENCES Epoch (epochID) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_ao_style FOREIGN KEY (styleID)
    REFERENCES Style (styleID) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_ao_gallery FOREIGN KEY (galleryID)
    REFERENCES Gallery (galleryID) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE Painting (
  objectID          CHAR(6)     NOT NULL,
  paintingMedium    VARCHAR(40) NOT NULL,            -- oil, watercolour, acrylic ...
  surfaceMaterialID SMALLINT UNSIGNED NULL,          -- "usually" painted on paper, canvas, wood
  PRIMARY KEY (objectID),
  CONSTRAINT fk_painting_object FOREIGN KEY (objectID)
    REFERENCES ArtObject (objectID) ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_painting_surface FOREIGN KEY (surfaceMaterialID)
    REFERENCES Material (materialID) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE Sculpture (
  objectID   CHAR(6)       NOT NULL,
  materialID SMALLINT UNSIGNED NOT NULL,
  heightCm   DECIMAL(7,2)  NOT NULL,
  weightKg   DECIMAL(9,2)  NOT NULL,
  PRIMARY KEY (objectID),
  CONSTRAINT ck_sculpture_size CHECK (heightCm > 0 AND weightKg > 0),
  CONSTRAINT fk_sculpture_object FOREIGN KEY (objectID)
    REFERENCES ArtObject (objectID) ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_sculpture_material FOREIGN KEY (materialID)
    REFERENCES Material (materialID) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE Statue (
  objectID   CHAR(6)       NOT NULL,
  materialID SMALLINT UNSIGNED NOT NULL,
  heightCm   DECIMAL(7,2)  NOT NULL,
  weightKg   DECIMAL(9,2)  NOT NULL,
  PRIMARY KEY (objectID),
  CONSTRAINT ck_statue_size CHECK (heightCm > 0 AND weightKg > 0),
  CONSTRAINT fk_statue_object FOREIGN KEY (objectID)
    REFERENCES ArtObject (objectID) ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_statue_material FOREIGN KEY (materialID)
    REFERENCES Material (materialID) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE OtherObject (
  objectID  CHAR(6)     NOT NULL,
  otherType VARCHAR(40) NOT NULL,                    -- print, photograph, drawing ...
  PRIMARY KEY (objectID),
  CONSTRAINT fk_other_object FOREIGN KEY (objectID)
    REFERENCES ArtObject (objectID) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE PermanentObject (
  objectID        CHAR(6)       NOT NULL,
  acquisitionDate DATE          NOT NULL,
```

```
  acquisitionCost DECIMAL(12,2) NOT NULL,            -- pounds sterling; 0.00 for gifts
  PRIMARY KEY (objectID),
  CONSTRAINT ck_permanent_cost CHECK (acquisitionCost >= 0),
  CONSTRAINT fk_permanent_object FOREIGN KEY (objectID)
    REFERENCES ArtObject (objectID) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE BorrowedObject (
  objectID CHAR(6) NOT NULL,
  PRIMARY KEY (objectID),
  CONSTRAINT fk_borrowed_object FOREIGN KEY (objectID)
    REFERENCES ArtObject (objectID) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE Borrowing (                           -- loans INTO this museum
  objectID     CHAR(6) NOT NULL,
  dateBorrowed DATE    NOT NULL,
  museumID     INT UNSIGNED NOT NULL,              -- the lender
  dateDue      DATE    NOT NULL,                   -- date to be returned
  dateReturned DATE    NULL,                       -- NULL = still held
  PRIMARY KEY (objectID, dateBorrowed),
  CONSTRAINT ck_borrowing_due CHECK (dateDue >= dateBorrowed),
  CONSTRAINT ck_borrowing_returned CHECK (dateReturned IS NULL OR dateReturned >= dateBorrowed),
  CONSTRAINT fk_borrowing_object FOREIGN KEY (objectID)
    REFERENCES BorrowedObject (objectID) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_borrowing_museum FOREIGN KEY (museumID)
    REFERENCES Museum (museumID) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE Lending (                             -- loans OUT of this museum
  objectID     CHAR(6) NOT NULL,
  dateLent     DATE    NOT NULL,
  museumID     INT UNSIGNED NOT NULL,              -- the borrower
  dateDue      DATE    NOT NULL,
  dateReturned DATE    NULL,                       -- NULL = still away
  PRIMARY KEY (objectID, dateLent),
  CONSTRAINT ck_lending_due CHECK (dateDue >= dateLent),
  CONSTRAINT ck_lending_returned CHECK (dateReturned IS NULL OR dateReturned >= dateLent),
  CONSTRAINT fk_lending_object FOREIGN KEY (objectID)
    REFERENCES PermanentObject (objectID) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_lending_museum FOREIGN KEY (museumID)
    REFERENCES Museum (museumID) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- ------------------------------------------------------------------------
-- E. Featured exhibitions: features, selected objects, replica sales
-- ------------------------------------------------------------------------
CREATE TABLE ExhibitionFeature (                   -- multi-valued composite attribute
  exhibitionID INT UNSIGNED NOT NULL,
  featureType  ENUM('ARTIST','CATEGORY','STYLE','EPOCH','ORIGIN') NOT NULL,
  featureValue VARCHAR(100) NOT NULL,              -- e.g. STYLE / Impressionist
  PRIMARY KEY (exhibitionID, featureType, featureValue),
  CONSTRAINT fk_exfeature_exhibition FOREIGN KEY (exhibitionID)
    REFERENCES FeaturedExhibition (exhibitionID) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE ExhibitionObject (                    -- objects used in a featured exhibition
  exhibitionID INT UNSIGNED NOT NULL,
  objectID     CHAR(6)      NOT NULL,
  PRIMARY KEY (exhibitionID, objectID),
  CONSTRAINT fk_exobject_exhibition FOREIGN KEY (exhibitionID)
    REFERENCES FeaturedExhibition (exhibitionID) ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_exobject_object FOREIGN KEY (objectID)
    REFERENCES ArtObject (objectID) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE ReplicaSale (                         -- replicas sold during an exhibition
  exhibitionID INT UNSIGNED NOT NULL,
  objectID     CHAR(6)      NOT NULL,
  unitPrice    DECIMAL(8,2) NOT NULL,
  quantitySold INT UNSIGNED NOT NULL DEFAULT 0,
  PRIMARY KEY (exhibitionID, objectID),
  CONSTRAINT ck_replica_price CHECK (unitPrice >= 0),
  -- a replica can only be sold for an object that is part of that exhibition
  CONSTRAINT fk_replica_exobject FOREIGN KEY (exhibitionID, objectID)
    REFERENCES ExhibitionObject (exhibitionID, objectID) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- ------------------------------------------------------------------------
-- F. Loyal visitors, preferences and invitations
-- ------------------------------------------------------------------------
CREATE TABLE LoyalVisitor (
  loyaltyNo CHAR(6)      NOT NULL,                 -- LMYYYY
  firstName VARCHAR(50)  NOT NULL,
  lastName  VARCHAR(50)  NOT NULL,
  address   VARCHAR(200) NOT NULL,
  email     VARCHAR(120) NULL,
  phone     VARCHAR(25)  NULL,
```

```
  joinDate  DATE         NOT NULL,
  PRIMARY KEY (loyaltyNo),
  UNIQUE KEY uq_visitor_email (email),
  CONSTRAINT ck_visitor_no CHECK (loyaltyNo REGEXP '^LM[0-9]{4}$')
) ENGINE=InnoDB;

CREATE TABLE VisitorPreference (                    -- multi-valued composite attribute
  loyaltyNo    CHAR(6)      NOT NULL,
  featureType  ENUM('ARTIST','CATEGORY','STYLE','EPOCH','ORIGIN') NOT NULL,
  featureValue VARCHAR(100) NOT NULL,
  PRIMARY KEY (loyaltyNo, featureType, featureValue),
  CONSTRAINT fk_pref_visitor FOREIGN KEY (loyaltyNo)
    REFERENCES LoyalVisitor (loyaltyNo) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE Invitation (
  loyaltyNo    CHAR(6)      NOT NULL,
  exhibitionID INT UNSIGNED NOT NULL,
  sentDate     DATE         NOT NULL,
  channel      ENUM('EMAIL','POST','SMS') NOT NULL,
  response     ENUM('PENDING','ACCEPTED','DECLINED') NOT NULL DEFAULT 'PENDING',
  PRIMARY KEY (loyaltyNo, exhibitionID),
  CONSTRAINT fk_invitation_visitor FOREIGN KEY (loyaltyNo)
    REFERENCES LoyalVisitor (loyaltyNo) ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_invitation_exhibition FOREIGN KEY (exhibitionID)
    REFERENCES FeaturedExhibition (exhibitionID) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- ------------------------------------------------------------------------------
-- G. Staff (generalisation: Staff -> Curator | Supervisor | other) and duties
-- ------------------------------------------------------------------------------
CREATE TABLE Staff (
  staffID   INT UNSIGNED NOT NULL AUTO_INCREMENT,
  firstName VARCHAR(50)  NOT NULL,
  lastName  VARCHAR(50)  NOT NULL,
  jobTitle  VARCHAR(60)  NOT NULL,
  staffType ENUM('CURATOR','SUPERVISOR','OTHER') NOT NULL,   -- discriminator
  phone     VARCHAR(25)  NULL,
  email     VARCHAR(120) NOT NULL,
  hireDate  DATE         NOT NULL,
  PRIMARY KEY (staffID),
  UNIQUE KEY uq_staff_email (email)
) ENGINE=InnoDB;

CREATE TABLE Curator (
  staffID        INT UNSIGNED NOT NULL,
  specialisation ENUM('PAINTING','SCULPTURE','STATUE','OTHER') NOT NULL,
  maxObjects     TINYINT UNSIGNED NOT NULL DEFAULT 20,       -- "certain number (say 20)"
  PRIMARY KEY (staffID),
  CONSTRAINT ck_curator_max CHECK (maxObjects BETWEEN 1 AND 100),
  CONSTRAINT fk_curator_staff FOREIGN KEY (staffID)
    REFERENCES Staff (staffID) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE Supervisor (
  staffID INT UNSIGNED NOT NULL,
  PRIMARY KEY (staffID),
  CONSTRAINT fk_supervisor_staff FOREIGN KEY (staffID)
    REFERENCES Staff (staffID) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE Maintains (                            -- curator <-> art object (M:N)
  staffID      INT UNSIGNED NOT NULL,
  objectID     CHAR(6)      NOT NULL,
  assignedDate DATE         NOT NULL,
  PRIMARY KEY (staffID, objectID),
  CONSTRAINT fk_maintains_curator FOREIGN KEY (staffID)
    REFERENCES Curator (staffID) ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_maintains_object FOREIGN KEY (objectID)
    REFERENCES ArtObject (objectID) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE SupervisesGallery (                    -- supervisor <-> gallery over time
  staffID   INT UNSIGNED NOT NULL,
  galleryID INT UNSIGNED NOT NULL,
  fromDate  DATE         NOT NULL,
  toDate    DATE         NULL,                      -- NULL = current allocation
  PRIMARY KEY (staffID, galleryID, fromDate),
  CONSTRAINT ck_supgallery_dates CHECK (toDate IS NULL OR toDate >= fromDate),
  CONSTRAINT fk_supgallery_supervisor FOREIGN KEY (staffID)
    REFERENCES Supervisor (staffID) ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_supgallery_gallery FOREIGN KEY (galleryID)
    REFERENCES Gallery (galleryID) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE SupervisesFeatured (                   -- supervisor <-> featured exhibition
  staffID      INT UNSIGNED NOT NULL,
  exhibitionID INT UNSIGNED NOT NULL,
```

```
  allocatedOn  DATE         NOT NULL,
  PRIMARY KEY (staffID, exhibitionID),
  CONSTRAINT fk_supfeatured_supervisor FOREIGN KEY (staffID)
    REFERENCES Supervisor (staffID) ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_supfeatured_exhibition FOREIGN KEY (exhibitionID)
    REFERENCES FeaturedExhibition (exhibitionID) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;
```

8.5 Views and triggers

```
-- ================================================================================
--  CS2019 Assessment 2 : Arts Museum Database
--  Script 3 of 4 : views and triggers
--  Triggers are installed after the sample data has been loaded.
--  (The mysql client and MySQL Workbench understand the DELIMITER command.)
-- ================================================================================
USE ArtsMuseumDB;

-- --------------------------------------------------------------------------------
-- Views
-- --------------------------------------------------------------------------------
-- What visitors may see next to an object in ANY exhibition: title, artist, year,
-- description, origin and epoch. Acquisition cost, acquisition date and loan
-- details are deliberately left out ("this information is not displayed").
CREATE OR REPLACE VIEW vw_ObjectDisplayInfo AS
SELECT ao.objectID, ao.title, a.artistName, ao.yearCreated, ao.description,
       o.originName AS countryOrCulture, e.epochName, s.styleName,
       ao.category, g.galleryName
FROM ArtObject ao
JOIN Origin o ON o.originID = ao.originID
JOIN Epoch  e ON e.epochID  = ao.epochID
JOIN Style  s ON s.styleID  = ao.styleID
LEFT JOIN Artist  a ON a.artistID  = ao.artistID
LEFT JOIN Gallery g ON g.galleryID = ao.galleryID;

-- Catalogue of every featured exhibition: only the public display information.
CREATE OR REPLACE VIEW vw_FeaturedExhibitionCatalogue AS
SELECT e.exhibitionName, fe.startDate, fe.endDate,
       d.objectID, d.title, d.artistName, d.yearCreated, d.description,
       d.countryOrCulture, d.epochName, d.category
FROM FeaturedExhibition fe
JOIN Exhibition e ON e.exhibitionID = fe.exhibitionID
JOIN ExhibitionObject eo ON eo.exhibitionID = fe.exhibitionID
JOIN vw_ObjectDisplayInfo d ON d.objectID = eo.objectID;

-- Objects that are borrowed and not yet returned (internal use only).
CREATE OR REPLACE VIEW vw_BorrowedObjectsHeld AS
SELECT ao.objectID, ao.title, m.museumName AS lender,
       b.dateBorrowed, b.dateDue
FROM Borrowing b
JOIN ArtObject ao ON ao.objectID = b.objectID
JOIN Museum m ON m.museumID = b.museumID
WHERE b.dateReturned IS NULL;

-- --------------------------------------------------------------------------------
-- Triggers: business rules that a CHECK or FOREIGN KEY cannot express because
-- they look at another table.
-- --------------------------------------------------------------------------------
DELIMITER $$

-- 1. A subclass row may only be attached to a superclass row of the right kind
--    (this keeps each generalisation disjoint).
CREATE TRIGGER trg_painting_bi BEFORE INSERT ON Painting FOR EACH ROW
BEGIN
  IF (SELECT category FROM ArtObject WHERE objectID = NEW.objectID) <> 'PAINTING' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Painting needs an ArtObject with category PAINTING';
  END IF;
END$$

CREATE TRIGGER trg_sculpture_bi BEFORE INSERT ON Sculpture FOR EACH ROW
BEGIN
  IF (SELECT category FROM ArtObject WHERE objectID = NEW.objectID) <> 'SCULPTURE' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Sculpture needs an ArtObject with category SCULPTURE';
  END IF;
END$$

CREATE TRIGGER trg_statue_bi BEFORE INSERT ON Statue FOR EACH ROW
BEGIN
  IF (SELECT category FROM ArtObject WHERE objectID = NEW.objectID) <> 'STATUE' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Statue needs an ArtObject with category STATUE';
  END IF;
END$$

CREATE TRIGGER trg_other_bi BEFORE INSERT ON OtherObject FOR EACH ROW
BEGIN
```

```
  IF (SELECT category FROM ArtObject WHERE objectID = NEW.objectID) <> 'OTHER' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'OtherObject needs an ArtObject with category OTHER';
  END IF;
END$$

CREATE TRIGGER trg_permanent_bi BEFORE INSERT ON PermanentObject FOR EACH ROW
BEGIN
  IF (SELECT ownership FROM ArtObject WHERE objectID = NEW.objectID) <> 'PERMANENT' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'PermanentObject needs an ArtObject with ownership PERMANENT';
  END IF;
END$$

CREATE TRIGGER trg_borrowed_bi BEFORE INSERT ON BorrowedObject FOR EACH ROW
BEGIN
  IF (SELECT ownership FROM ArtObject WHERE objectID = NEW.objectID) <> 'BORROWED' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'BorrowedObject needs an ArtObject with ownership BORROWED';
  END IF;
END$$

CREATE TRIGGER trg_regular_bi BEFORE INSERT ON RegularExhibition FOR EACH ROW
BEGIN
  IF (SELECT exhibitionKind FROM Exhibition WHERE exhibitionID = NEW.exhibitionID) <> 'REGULAR' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'RegularExhibition needs an Exhibition of kind REGULAR';
  END IF;
END$$

CREATE TRIGGER trg_featured_bi BEFORE INSERT ON FeaturedExhibition FOR EACH ROW
BEGIN
  IF (SELECT exhibitionKind FROM Exhibition WHERE exhibitionID = NEW.exhibitionID) <> 'FEATURED' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'FeaturedExhibition needs an Exhibition of kind FEATURED';
  END IF;
END$$

CREATE TRIGGER trg_curator_bi BEFORE INSERT ON Curator FOR EACH ROW
BEGIN
  IF (SELECT staffType FROM Staff WHERE staffID = NEW.staffID) <> 'CURATOR' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Curator needs a Staff member of type CURATOR';
  END IF;
END$$

CREATE TRIGGER trg_supervisor_bi BEFORE INSERT ON Supervisor FOR EACH ROW
BEGIN
  IF (SELECT staffType FROM Staff WHERE staffID = NEW.staffID) <> 'SUPERVISOR' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Supervisor needs a Staff member of type SUPERVISOR';
  END IF;
END$$

-- 2. An object is ON_DISPLAY exactly when it hangs in a gallery; category and
--    ownership are fixed once an object has been registered.
CREATE TRIGGER trg_artobject_bi BEFORE INSERT ON ArtObject FOR EACH ROW
BEGIN
  IF (NEW.status = 'ON_DISPLAY') <> (NEW.galleryID IS NOT NULL) THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'ON_DISPLAY objects need a gallery; any other status must have no gallery';
  END IF;
END$$

CREATE TRIGGER trg_artobject_bu BEFORE UPDATE ON ArtObject FOR EACH ROW
BEGIN
  IF NEW.category <> OLD.category OR NEW.ownership <> OLD.ownership THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'category and ownership cannot be changed after registration';
  END IF;
  IF (NEW.status = 'ON_DISPLAY') <> (NEW.galleryID IS NOT NULL) THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'ON_DISPLAY objects need a gallery; any other status must have no gallery';
  END IF;
END$$

-- 3. A curator maintains only objects of his or her specialisation and never
--    more than maxObjects objects.
CREATE TRIGGER trg_maintains_bi BEFORE INSERT ON Maintains FOR EACH ROW
BEGIN
  DECLARE v_spec VARCHAR(10);
  DECLARE v_max  INT;
  DECLARE v_cat  VARCHAR(10);
  DECLARE v_cnt  INT;
  SELECT specialisation, maxObjects INTO v_spec, v_max
    FROM Curator WHERE staffID = NEW.staffID;
  SELECT category INTO v_cat FROM ArtObject WHERE objectID = NEW.objectID;
  IF v_spec <> v_cat THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Curator specialisation does not match the art object category';
  END IF;
  SELECT COUNT(*) INTO v_cnt FROM Maintains WHERE staffID = NEW.staffID;
  IF v_cnt >= v_max THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Curator already maintains the maximum number of art objects';
```

```
  END IF;
END$$

CREATE TRIGGER trg_maintains_bu BEFORE UPDATE ON Maintains FOR EACH ROW
BEGIN
  DECLARE v_spec VARCHAR(10);
  DECLARE v_max  INT;
  DECLARE v_cat  VARCHAR(10);
  DECLARE v_cnt  INT;
  SELECT specialisation, maxObjects INTO v_spec, v_max
    FROM Curator WHERE staffID = NEW.staffID;
  SELECT category INTO v_cat FROM ArtObject WHERE objectID = NEW.objectID;
  IF v_spec <> v_cat THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Curator specialisation does not match the art object category';
  END IF;
  SELECT COUNT(*) INTO v_cnt FROM Maintains
   WHERE staffID = NEW.staffID
     AND NOT (staffID = OLD.staffID AND objectID = OLD.objectID);
  IF v_cnt >= v_max THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Curator already maintains the maximum number of art objects';
  END IF;
END$$

DELIMITER ;
```

8.6 Known limits

- CHECK constraints cannot look at other tables, which is why the cross-table rules are triggers.
- Participation that must be *mandatory* from the parent side (every ArtObject has a subclass row; every borrowed object has a loan) cannot be declared; transaction T1 writes the rows together and the test script verifies them.
- The status of an object and its loan records are changed by staff; the database refuses contradictions between status and gallery, but does not copy a new loan into the status automatically.

Task 9 – Sample data

The script `02_data.sql` fills all 31 tables; **every table has more than five rows** (310 rows in total). The rows are inserted parents first, so every foreign key is satisfied. The data is deliberately varied so that every transaction in Appendix A returns something meaningful.

Table	Rows	Table	Rows
Origin	8	BorrowedObject	6
Epoch	7	Borrowing	7
Style	8	Lending	7
Material	8	ExhibitionFeature	9
Museum	7	ExhibitionObject	14
Artist	10	ReplicaSale	10
Exhibition	12	LoyalVisitor	8
RegularExhibition	6	VisitorPreference	16
FeaturedExhibition	6	Invitation	11
Gallery	8	Staff	16
ArtObject	24	Curator	8
Painting	6	Supervisor	6
Sculpture	6	Maintains	26
Statue	6	SupervisesGallery	10
OtherObject	6	SupervisesFeatured	10
PermanentObject	18		
Total	**310**		

What the data demonstrates.

- 24 art objects: 6 paintings, 6 sculptures, 6 statues and 6 others; 18 permanent and 6 borrowed; objects with an unknown artist (AO0010, AO0011, AO0013, AO0014, AO0017) and with a BCE year (AO0010, AO0013, AO0014).
- Loan history: AO0006 was borrowed twice from the same museum (once returned, once held); AO0018 has been returned (status RETURNED); AO0012 and AO0024 are lent out now (status ON_LOAN); five earlier lendings are complete.
- A living artist (Hiroshi Tanaka, no death date) next to historical artists, including a name with an accent.
- Twelve exhibitions: six regular (with eight galleries) and six featured, some past, some current, some in the future; one featured exhibition (Marchetti: A Baroque Life) has no invitations yet, to show why transaction T9 uses an outer join.
- Curators with different workload limits, objects maintained by two curators (AO0003, AO0009, AO0013), supervisors whose allocations have ended, visitors without an email address or without a phone number.

9.1 INSERT INTO statements

```
-- ========================================================================
--  CS2019 Assessment 2 : Arts Museum Database
--  Script 2 of 4 : sample data (every table has more than five rows)
--  Rows are inserted parents-first so that every foreign key is satisfied.
-- ========================================================================
USE ArtsMuseumDB;
SET NAMES utf8mb4;

-- Origin (8 rows)
INSERT INTO Origin (originID, originName) VALUES
  (1, 'Italian'),
  (2, 'Greek'),
  (3, 'Indian'),
  (4, 'French'),
  (5, 'Dutch'),
  (6, 'British'),
  (7, 'Spanish'),
```

```
  (8, 'Japanese');

-- Epoch (7 rows)
INSERT INTO Epoch (epochID, epochName) VALUES
  (1, 'Ancient'),
  (2, 'Medieval'),
  (3, 'Renaissance'),
  (4, 'Baroque'),
  (5, 'Edo Period'),
  (6, 'Modern'),
  (7, 'Contemporary');

-- Style (8 rows)
INSERT INTO Style (styleID, styleName) VALUES
  (1, 'Classic'),
  (2, 'Realist'),
  (3, 'Baroque'),
  (4, 'Impressionist'),
  (5, 'Cubist'),
  (6, 'Abstract'),
  (7, 'Modernist'),
  (8, 'Ukiyo-e');

-- Material (8 rows)
INSERT INTO Material (materialID, materialName) VALUES
  (1, 'Canvas'),
  (2, 'Paper'),
  (3, 'Wood'),
  (4, 'Marble'),
  (5, 'Bronze'),
  (6, 'Limestone'),
  (7, 'Terracotta'),
  (8, 'Plaster');

-- Museum (7 rows)
INSERT INTO Museum (museumID, museumName, description, address, phone, contactName) VALUES
  (1, 'Riverside Museum of Fine Art',
    'Tay waterfront museum with strong Dutch and French collections.',
    '21 Quayside Walk, Dundee DD1 4QA', '01632 960211', 'Fiona McKenzie'),
  (2, 'Highland Heritage Gallery', 'Regional gallery of Highland and medieval art.',
    '5 Castle Wynd, Inverness IV2 3EE', '01632 960322', 'Callum Fraser'),
  (3, 'Thistle Modern Art Centre', 'Centre for modern and contemporary art.',
    '88 Sauchiehall Lane, Glasgow G2 3AB', '0141 496 0433', 'Priya Nair'),
  (4, 'Tay Valley Art Museum', 'Museum of seventeenth to nineteenth century painting.',
    '3 Kinnoull Terrace, Perth PH2 7BT', '01632 960544', 'Douglas Reid'),
  (5, 'Forth Maritime Arts Museum', 'Art of harbours, ships and the sea.',
    '14 Shore Road, Edinburgh EH6 6QW', '0131 496 0655', 'Eilidh Stewart'),
  (6, 'Castlegate Sculpture Museum', 'Specialist museum of sculpture and statuary.',
    '9 Upper Bridge Street, Stirling FK8 1AA', '01632 960766', 'Martin Gallagher'),
  (7, 'Solway Print and Photography Museum', 'Collection of prints, drawings and photographs.',
    '27 High Street, Dumfries DG1 2JD', '01632 960877', 'Hannah Brodie');

-- Artist (10 rows)
INSERT INTO Artist (artistID, artistName, dateBorn, dateDied, originID, epochID, mainStyleID, description) VALUES
  (1, 'Giulia Marchetti', '1601-03-12', '1672-09-04', 1, 4, 3,
    'Painter of religious scenes in the Roman Baroque tradition.'),
  (2, 'Pieter van der Linden', '1620-05-21', '1689-11-30', 5, 4, 2,
    'Dutch painter of harbours and merchant portraits.'),
  (3, 'Élodie Vasseur', '1845-07-09', '1919-02-17', 4, 6, 4,
    'Painter of domestic interiors and everyday life.'),
  (4, 'Rajesh Varma', '1932-11-02', '2004-06-19', 3, 6, 6,
    'Pioneer of geometric abstraction in post-independence India.'),
  (5, 'Hiroshi Tanaka', '1958-04-23', NULL, 8, 7, 7,
    'Living sculptor working in bronze and stone.'),
  (6, 'Margaret Ellison', '1903-08-14', '1988-01-30', 6, 6, 7,
    'British sculptor known for organic wood and bronze figures.'),
  (7, 'Carlos Ibarra Soler', '1889-10-02', '1971-12-11', 7, 6, 5,
    'Spanish sculptor and draughtsman of the Cubist circle.'),
  (8, 'Tomasso Berardi', '1470-02-11', '1541-08-23', 1, 3, 1,
    'Florentine sculptor of marble and bronze figures.'),
  (9, 'Shunpei Katsura', '1771-04-05', '1844-09-12', 8, 5, 8,
    'Japanese woodblock printmaker of the Edo period.'),
  (10, 'Isla MacRae', '1901-06-03', '1984-02-21', 6, 6, 2,
    'Scottish documentary photographer of harbours and street life.');

-- Exhibition (12 rows)
INSERT INTO Exhibition (exhibitionID, exhibitionName, description, exhibitionKind) VALUES
  (1, 'European Masters',
    'Permanent display of European painting from the sixteenth to nineteenth centuries.',
    'REGULAR'),
  (2, 'Ancient Worlds', 'Greek, Roman and early Mediterranean art.', 'REGULAR'),
  (3, 'Sculpture Court', 'Permanent display of sculpture and statuary.', 'REGULAR'),
  (4, 'Modern and Contemporary', 'Twentieth and twenty-first century art.', 'REGULAR'),
  (5, 'Asian Art', 'Art of India, Japan and East Asia.', 'REGULAR'),
  (6, 'Prints and Photography', 'Works on paper and photographic prints.', 'REGULAR'),
  (7, 'Light and Water: The Impressionist Vision',
    'Impressionist paintings of light, water and everyday life.', 'FEATURED'),
  (8, 'Gods and Heroes in Marble',
```

```
    'Marble statues of gods and heroes from Greece and Renaissance Italy.', 'FEATURED'),
  (9, 'The Dutch Golden Age', 'Dutch Baroque harbours, portraits and still life.', 'FEATURED'),
  (10, 'Modern Abstractions', 'Abstract art of the twentieth and twenty-first centuries.',
    'FEATURED'),
  (11, 'Prints of the Floating World', 'Japanese woodblock prints of the Edo period.', 'FEATURED'),
  (12, 'Marchetti: A Baroque Life', 'Paintings and drawings by Giulia Marchetti.', 'FEATURED');

-- RegularExhibition (6 rows)
INSERT INTO RegularExhibition (exhibitionID) VALUES
  (1),
  (2),
  (3),
  (4),
  (5),
  (6);

-- FeaturedExhibition (6 rows)
INSERT INTO FeaturedExhibition (exhibitionID, startDate, endDate) VALUES
  (7, '2026-03-01', '2026-06-30'),
  (8, '2026-05-15', '2026-09-30'),
  (9, '2026-09-01', '2026-12-31'),
  (10, '2026-10-10', '2027-01-31'),
  (11, '2027-02-15', '2027-05-30'),
  (12, '2027-03-01', '2027-07-31');

-- Gallery (8 rows)
INSERT INTO Gallery (galleryID, galleryName, floorLevel, regularExhibitionID) VALUES
  (1, 'Baroque Hall', 1, 1),
  (2, 'Impressionist Room', 1, 1),
  (3, 'Hellenic Gallery', 0, 2),
  (4, 'Marble Court', 0, 3),
  (5, 'Bronze Gallery', 0, 3),
  (6, 'Abstract Gallery', 2, 4),
  (7, 'Japanese Print Room', 2, 5),
  (8, 'Works on Paper Room', 1, 6);

-- ArtObject (24 rows)
INSERT INTO ArtObject (objectID, title, description, yearCreated, category, ownership, status, artistID, originID,
epochID, styleID, galleryID) VALUES
  ('AO0001', 'Harbour at Dawn', 'Fishing boats leave a misty Dutch harbour at first light.',
    1655, 'PAINTING', 'PERMANENT', 'ON_DISPLAY', 2, 5, 4, 2, 1),
  ('AO0002', 'The Seamstress',
    'A woman sews by a sunlit window, painted in loose, feathery brushwork.', 1889, 'PAINTING',
    'PERMANENT', 'ON_DISPLAY', 3, 4, 6, 4, 2),
  ('AO0003', 'Saint Cecilia at the Organ', 'The patron saint of music, lit by a single candle.',
    1638, 'PAINTING', 'PERMANENT', 'ON_DISPLAY', 1, 1, 4, 3, 1),
  ('AO0004', 'Monsoon Geometry No. 4',
    'Interlocking blue and ochre forms evoking monsoon skies.', 1968, 'PAINTING', 'PERMANENT',
    'ON_DISPLAY', 4, 3, 6, 6, 6),
  ('AO0005', 'Portrait of a Merchant''s Wife',
    'Half-length portrait of a Dutch merchant''s wife in black silk.', 1662, 'PAINTING',
    'BORROWED', 'ON_DISPLAY', 2, 5, 4, 2, 1),
  ('AO0006', 'Winter Orchard', 'Delicate watercolour of an orchard under frost.', 1894,
    'PAINTING', 'BORROWED', 'IN_STORE', 3, 4, 6, 4, NULL),
  ('AO0007', 'Reclining Figure in Elm', 'Carved elm figure with smooth, flowing curves.', 1952,
    'SCULPTURE', 'PERMANENT', 'ON_DISPLAY', 6, 6, 6, 7, 4),
  ('AO0008', 'Mother and Child (Study)', 'Bronze study for a larger public commission.', 1949,
    'SCULPTURE', 'PERMANENT', 'UNDER_RESTORATION', 6, 6, 6, 7, NULL),
  ('AO0009', 'Wave Form', 'Abstract limestone sculpture suggesting a breaking wave.', 2011,
    'SCULPTURE', 'PERMANENT', 'ON_DISPLAY', 5, 8, 7, 6, 5),
  ('AO0010', 'Horse Head Fragment',
    'Marble fragment from a Greek horse sculpture; maker unknown.', -450, 'SCULPTURE',
    'PERMANENT', 'IN_STORE', NULL, 2, 1, 1, NULL),
  ('AO0011', 'Carved Temple Guardian', 'Guardian figure from a medieval Indian temple doorway.',
    1150, 'SCULPTURE', 'BORROWED', 'ON_DISPLAY', NULL, 3, 2, 1, 5),
  ('AO0012', 'Torso of Flight', 'Cubist bronze torso built from angular planes.', 1925,
    'SCULPTURE', 'PERMANENT', 'ON_LOAN', 7, 7, 6, 5, NULL),
  ('AO0013', 'Statue of Athena', 'Marble statue of the goddess Athena in armour.', -325,
    'STATUE', 'PERMANENT', 'ON_DISPLAY', NULL, 2, 1, 1, 3),
  ('AO0014', 'Kouros Figure', 'Standing marble youth in the archaic Greek manner.', -540,
    'STATUE', 'PERMANENT', 'ON_DISPLAY', NULL, 2, 1, 1, 3),
  ('AO0015', 'Youth with a Lyre', 'Renaissance marble of a young musician.', 1512, 'STATUE',
    'PERMANENT', 'ON_DISPLAY', 8, 1, 3, 1, 4),
  ('AO0016', 'Saint Michael and the Dragon',
    'Bronze statue of the archangel defeating the dragon.', 1520, 'STATUE', 'BORROWED',
    'ON_DISPLAY', 8, 1, 3, 1, 4),
  ('AO0017', 'Seated Bodhisattva', 'Gilded wooden seated figure from a Japanese temple.', 1200,
    'STATUE', 'PERMANENT', 'ON_DISPLAY', NULL, 8, 2, 1, 7),
  ('AO0018', 'The Founder (Monument Maquette)', 'Bronze maquette for a civic monument.', 1958,
    'STATUE', 'BORROWED', 'RETURNED', 6, 6, 6, 2, NULL),
  ('AO0019', 'Fishing Boats Beneath the Wave',
    'Colour woodblock print of boats under a towering wave.', 1831, 'OTHER', 'PERMANENT',
    'ON_DISPLAY', 9, 8, 5, 8, 7),
  ('AO0020', 'Moonlit Ferry at Sumida', 'Night-time woodblock print of a river ferry.', 1835,
    'OTHER', 'BORROWED', 'ON_DISPLAY', 9, 8, 5, 8, 7),
  ('AO0021', 'Mill Street, 1932', 'Silver gelatin photograph of a granite-lined city street.',
    1932, 'OTHER', 'PERMANENT', 'ON_DISPLAY', 10, 6, 6, 2, 8),
  ('AO0022', 'Harbour Cranes at Dusk', 'Photograph of harbour cranes silhouetted at dusk.', 1938,
```

```
    'OTHER', 'PERMANENT', 'ON_DISPLAY', 10, 6, 6, 2, 8),
  ('AO0023', 'Study for The Sacrifice of Isaac',
    'Red chalk drawing, a preparatory study for a lost altarpiece.', 1636, 'OTHER',
    'PERMANENT', 'IN_STORE', 1, 1, 4, 3, NULL),
  ('AO0024', 'Lotus Pond at Night', 'Abstract lithograph of lotus leaves in dark water.', 1974,
    'OTHER', 'PERMANENT', 'ON_LOAN', 4, 3, 6, 6, NULL);

-- Painting (6 rows)
INSERT INTO Painting (objectID, paintingMedium, surfaceMaterialID) VALUES
  ('AO0001', 'Oil', 1),
  ('AO0002', 'Oil', 1),
  ('AO0003', 'Oil', 1),
  ('AO0004', 'Acrylic', 1),
  ('AO0005', 'Oil', 3),
  ('AO0006', 'Watercolour', 2);

-- Sculpture (6 rows)
INSERT INTO Sculpture (objectID, materialID, heightCm, weightKg) VALUES
  ('AO0007', 3, 120.00, 85.50),
  ('AO0008', 5, 64.00, 40.00),
  ('AO0009', 6, 150.00, 620.00),
  ('AO0010', 4, 38.00, 55.00),
  ('AO0011', 6, 140.00, 480.00),
  ('AO0012', 5, 95.00, 70.00);

-- Statue (6 rows)
INSERT INTO Statue (objectID, materialID, heightCm, weightKg) VALUES
  ('AO0013', 4, 182.00, 410.00),
  ('AO0014', 4, 195.00, 380.00),
  ('AO0015', 4, 160.00, 290.00),
  ('AO0016', 5, 110.00, 150.00),
  ('AO0017', 3, 98.00, 60.00),
  ('AO0018', 5, 85.00, 120.00);

-- OtherObject (6 rows)
INSERT INTO OtherObject (objectID, otherType) VALUES
  ('AO0019', 'Woodblock print'),
  ('AO0020', 'Woodblock print'),
  ('AO0021', 'Photograph'),
  ('AO0022', 'Photograph'),
  ('AO0023', 'Chalk drawing'),
  ('AO0024', 'Lithograph');

-- PermanentObject (18 rows)
INSERT INTO PermanentObject (objectID, acquisitionDate, acquisitionCost) VALUES
  ('AO0001', '2008-03-14', 125000.00),
  ('AO0002', '2011-09-02', 98000.00),
  ('AO0003', '2003-06-20', 210000.00),
  ('AO0004', '2015-11-30', 54000.00),
  ('AO0007', '1999-05-11', 76000.00),
  ('AO0008', '2001-02-19', 48000.00),
  ('AO0009', '2013-10-08', 88000.00),
  ('AO0010', '1987-07-23', 15000.00),
  ('AO0012', '2006-04-04', 132000.00),
  ('AO0013', '1992-09-01', 340000.00),
  ('AO0014', '1992-09-01', 295000.00),
  ('AO0015', '2005-03-17', 410000.00),
  ('AO0017', '2010-12-12', 185000.00),
  ('AO0019', '2014-01-25', 32000.00),
  ('AO0021', '2017-06-09', 2800.00),
  ('AO0022', '2017-06-09', 3100.00),
  ('AO0023', '2003-06-20', 18000.00),
  ('AO0024', '2016-02-15', 7500.00);

-- BorrowedObject (6 rows)
INSERT INTO BorrowedObject (objectID) VALUES
  ('AO0005'),
  ('AO0006'),
  ('AO0011'),
  ('AO0016'),
  ('AO0018'),
  ('AO0020');

-- Borrowing (7 rows)
INSERT INTO Borrowing (objectID, dateBorrowed, museumID, dateDue, dateReturned) VALUES
  ('AO0005', '2026-03-01', 1, '2027-03-01', NULL),
  ('AO0006', '2025-02-01', 4, '2025-08-01', '2025-07-28'),
  ('AO0006', '2026-05-01', 4, '2027-05-01', NULL),
  ('AO0011', '2026-01-15', 2, '2027-01-15', NULL),
  ('AO0016', '2026-04-10', 6, '2027-04-10', NULL),
  ('AO0018', '2025-06-01', 6, '2026-06-01', '2026-05-28'),
  ('AO0020', '2026-09-01', 7, '2027-06-30', NULL);

-- Lending (7 rows)
INSERT INTO Lending (objectID, dateLent, museumID, dateDue, dateReturned) VALUES
  ('AO0012', '2026-09-15', 3, '2027-03-15', NULL),
  ('AO0024', '2026-10-01', 7, '2027-01-31', NULL),
  ('AO0003', '2024-04-01', 2, '2024-10-01', '2024-09-28'),
```

```
  ('AO0007', '2025-01-10', 3, '2025-07-10', '2025-07-09'),
  ('AO0015', '2025-05-05', 6, '2025-11-05', '2025-11-01'),
  ('AO0019', '2025-09-01', 7, '2026-02-28', '2026-02-25'),
  ('AO0001', '2023-06-01', 5, '2023-12-01', '2023-11-30');

-- ExhibitionFeature (9 rows)
INSERT INTO ExhibitionFeature (exhibitionID, featureType, featureValue) VALUES
  (7, 'STYLE', 'Impressionist'),
  (8, 'CATEGORY', 'STATUE'),
  (8, 'ORIGIN', 'Greek'),
  (9, 'EPOCH', 'Baroque'),
  (9, 'ORIGIN', 'Dutch'),
  (10, 'STYLE', 'Abstract'),
  (11, 'CATEGORY', 'OTHER'),
  (11, 'ORIGIN', 'Japanese'),
  (12, 'ARTIST', 'Giulia Marchetti');

-- ExhibitionObject (14 rows)
INSERT INTO ExhibitionObject (exhibitionID, objectID) VALUES
  (7, 'AO0002'),
  (7, 'AO0006'),
  (8, 'AO0010'),
  (8, 'AO0013'),
  (8, 'AO0014'),
  (8, 'AO0015'),
  (9, 'AO0001'),
  (9, 'AO0005'),
  (10, 'AO0004'),
  (10, 'AO0009'),
  (11, 'AO0019'),
  (11, 'AO0020'),
  (12, 'AO0003'),
  (12, 'AO0023');

-- ReplicaSale (10 rows)
INSERT INTO ReplicaSale (exhibitionID, objectID, unitPrice, quantitySold) VALUES
  (7, 'AO0002', 45.00, 120),
  (7, 'AO0006', 30.00, 64),
  (8, 'AO0013', 85.00, 40),
  (8, 'AO0014', 85.00, 33),
  (8, 'AO0015', 120.00, 25),
  (9, 'AO0001', 40.00, 150),
  (9, 'AO0005', 40.00, 96),
  (10, 'AO0004', 55.00, 18),
  (11, 'AO0019', 25.00, 0),
  (12, 'AO0003', 60.00, 0);

-- LoyalVisitor (8 rows)
INSERT INTO LoyalVisitor (loyaltyNo, firstName, lastName, address, email, phone, joinDate) VALUES
  ('LM1001', 'Catherine', 'Bell', '22 Rosemount Place, Aberdeen AB25 2QX',
    'catherine.bell@example.org', '07700 900101', '2019-03-12'),
  ('LM1002', 'James', 'Okoro', '8 Holburn Court, Aberdeen AB10 6QQ', 'james.okoro@example.org',
    '07700 900102', '2020-01-20'),
  ('LM1003', 'Sophie', 'Laurent', '15 Ferryhill Road, Aberdeen AB11 6QN',
    'sophie.laurent@example.org', NULL, '2018-09-05'),
  ('LM1004', 'Ravi', 'Patel', '3 Westburn Drive, Aberdeen AB25 2QZ', 'ravi.patel@example.org',
    '07700 900104', '2021-06-14'),
  ('LM1005', 'Mei Lin', 'Chen', '41 Cults Avenue, Aberdeen AB15 9QP', 'meilin.chen@example.org',
    '07700 900105', '2022-02-28'),
  ('LM1006', 'Angus', 'Murray', '6 Kingswells Brae, Aberdeen AB15 8QT',
    'angus.murray@example.org', NULL, '2017-11-09'),
  ('LM1007', 'Elena', 'Rossi', '19 Seaton Walk, Aberdeen AB24 1QS', 'elena.rossi@example.org',
    '07700 900107', '2023-04-03'),
  ('LM1008', 'David', 'Walker', '27 Bridge of Don Road, Aberdeen AB23 8QH', NULL, '07700 900108',
    '2024-08-17');

-- VisitorPreference (16 rows)
INSERT INTO VisitorPreference (loyaltyNo, featureType, featureValue) VALUES
  ('LM1001', 'STYLE', 'Impressionist'),
  ('LM1001', 'ORIGIN', 'French'),
  ('LM1002', 'CATEGORY', 'STATUE'),
  ('LM1002', 'ORIGIN', 'Greek'),
  ('LM1003', 'EPOCH', 'Baroque'),
  ('LM1003', 'ORIGIN', 'Dutch'),
  ('LM1003', 'ARTIST', 'Giulia Marchetti'),
  ('LM1004', 'STYLE', 'Abstract'),
  ('LM1004', 'CATEGORY', 'OTHER'),
  ('LM1005', 'ORIGIN', 'Japanese'),
  ('LM1005', 'CATEGORY', 'OTHER'),
  ('LM1006', 'ARTIST', 'Margaret Ellison'),
  ('LM1006', 'CATEGORY', 'SCULPTURE'),
  ('LM1007', 'STYLE', 'Impressionist'),
  ('LM1008', 'EPOCH', 'Modern'),
  ('LM1008', 'STYLE', 'Abstract');

-- Invitation (11 rows)
INSERT INTO Invitation (loyaltyNo, exhibitionID, sentDate, channel, response) VALUES
  ('LM1001', 7, '2026-02-10', 'EMAIL', 'ACCEPTED'),
```

```
  ('LM1007', 7, '2026-02-10', 'EMAIL', 'DECLINED'),
  ('LM1002', 8, '2026-04-28', 'POST', 'ACCEPTED'),
  ('LM1006', 8, '2026-04-28', 'EMAIL', 'DECLINED'),
  ('LM1003', 9, '2026-08-20', 'EMAIL', 'ACCEPTED'),
  ('LM1008', 9, '2026-08-20', 'SMS', 'DECLINED'),
  ('LM1001', 9, '2026-08-20', 'POST', 'DECLINED'),
  ('LM1004', 10, '2026-09-28', 'EMAIL', 'ACCEPTED'),
  ('LM1008', 10, '2026-09-28', 'SMS', 'ACCEPTED'),
  ('LM1005', 11, '2026-10-25', 'EMAIL', 'PENDING'),
  ('LM1004', 11, '2026-10-25', 'EMAIL', 'PENDING');

-- Staff (16 rows)
INSERT INTO Staff (staffID, firstName, lastName, jobTitle, staffType, phone, email, hireDate) VALUES
  (1, 'Eleanor', 'Whitfield', 'Chief Curator', 'CURATOR', '0131 496 0101',
    'eleanor.whitfield@museum.example', '2009-02-02'),
  (2, 'Marcus', 'Okafor', 'Curator of Paintings', 'CURATOR', '0131 496 0102',
    'marcus.okafor@museum.example', '2014-09-15'),
  (3, 'Sofia', 'Lindqvist', 'Curator of Sculpture', 'CURATOR', '0131 496 0103',
    'sofia.lindqvist@museum.example', '2016-04-04'),
  (4, 'Daniel', 'Ruiz', 'Senior Curator of Sculpture', 'CURATOR', '0131 496 0104',
    'daniel.ruiz@museum.example', '2011-01-10'),
  (5, 'Aisha', 'Rahman', 'Curator of Antiquities', 'CURATOR', '0131 496 0105',
    'aisha.rahman@museum.example', '2018-06-18'),
  (6, 'Tom', 'Henderson', 'Curator of Statuary', 'CURATOR', '0131 496 0106',
    'tom.henderson@museum.example', '2012-10-01'),
  (7, 'Yuki', 'Matsuda', 'Curator of Prints and Photography', 'CURATOR', '0131 496 0107',
    'yuki.matsuda@museum.example', '2019-03-25'),
  (8, 'Grace', 'Campbell', 'Curator of Works on Paper', 'CURATOR', NULL,
    'grace.campbell@museum.example', '2015-05-11'),
  (9, 'Hamish', 'Gordon', 'Gallery Supervisor', 'SUPERVISOR', '07700 900209',
    'hamish.gordon@museum.example', '2017-02-06'),
  (10, 'Isabelle', 'Moreau', 'Gallery Supervisor', 'SUPERVISOR', '07700 900210',
    'isabelle.moreau@museum.example', '2019-07-22'),
  (11, 'Kwame', 'Mensah', 'Senior Gallery Supervisor', 'SUPERVISOR', '07700 900211',
    'kwame.mensah@museum.example', '2013-11-04'),
  (12, 'Rhona', 'MacLeod', 'Gallery Supervisor', 'SUPERVISOR', '07700 900212',
    'rhona.macleod@museum.example', '2021-01-18'),
  (13, 'Oliver', 'Grant', 'Exhibition Supervisor', 'SUPERVISOR', '07700 900213',
    'oliver.grant@museum.example', '2016-08-30'),
  (14, 'Nadia', 'Petrov', 'Gallery Supervisor', 'SUPERVISOR', NULL,
    'nadia.petrov@museum.example', '2022-05-09'),
  (15, 'Fraser', 'Duncan', 'Head of Security', 'OTHER', '0131 496 0115',
    'fraser.duncan@museum.example', '2010-03-15'),
  (16, 'Leila', 'Hosseini', 'Visitor Services Manager', 'OTHER', '0131 496 0116',
    'leila.hosseini@museum.example', '2018-01-08');

-- Curator (8 rows)
INSERT INTO Curator (staffID, specialisation, maxObjects) VALUES
  (1, 'PAINTING', 20),
  (2, 'PAINTING', 8),
  (3, 'SCULPTURE', 6),
  (4, 'SCULPTURE', 20),
  (5, 'STATUE', 6),
  (6, 'STATUE', 10),
  (7, 'OTHER', 12),
  (8, 'OTHER', 20);

-- Supervisor (6 rows)
INSERT INTO Supervisor (staffID) VALUES
  (9),
  (10),
  (11),
  (12),
  (13),
  (14);

-- Maintains (26 rows)
INSERT INTO Maintains (staffID, objectID, assignedDate) VALUES
  (1, 'AO0001', '2018-01-15'),
  (1, 'AO0003', '2018-01-15'),
  (2, 'AO0003', '2019-02-01'),
  (2, 'AO0002', '2019-02-01'),
  (2, 'AO0004', '2019-02-01'),
  (2, 'AO0005', '2026-03-01'),
  (1, 'AO0006', '2026-05-01'),
  (3, 'AO0007', '2017-05-10'),
  (3, 'AO0008', '2017-05-10'),
  (3, 'AO0009', '2017-05-10'),
  (4, 'AO0009', '2013-10-08'),
  (4, 'AO0010', '2012-03-01'),
  (4, 'AO0011', '2026-01-15'),
  (3, 'AO0012', '2017-05-10'),
  (5, 'AO0013', '2018-09-03'),
  (6, 'AO0013', '2012-10-01'),
  (5, 'AO0014', '2018-09-03'),
  (5, 'AO0015', '2018-09-03'),
  (6, 'AO0016', '2026-04-10'),
  (6, 'AO0017', '2012-10-01'),
```

```
  (7, 'AO0019', '2019-04-01'),
  (7, 'AO0020', '2026-09-01'),
  (8, 'AO0021', '2017-06-12'),
  (8, 'AO0022', '2017-06-12'),
  (7, 'AO0023', '2019-04-01'),
  (8, 'AO0024', '2016-02-15');

-- SupervisesGallery (10 rows)
INSERT INTO SupervisesGallery (staffID, galleryID, fromDate, toDate) VALUES
  (9, 1, '2025-01-06', NULL),
  (10, 2, '2025-01-06', NULL),
  (11, 3, '2024-09-01', NULL),
  (12, 4, '2025-03-03', NULL),
  (12, 5, '2025-03-03', NULL),
  (14, 6, '2025-06-02', NULL),
  (13, 7, '2025-06-02', '2026-03-31'),
  (10, 7, '2026-04-01', NULL),
  (11, 8, '2025-01-06', NULL),
  (9, 2, '2023-05-01', '2024-12-31');

-- SupervisesFeatured (10 rows)
INSERT INTO SupervisesFeatured (staffID, exhibitionID, allocatedOn) VALUES
  (13, 7, '2026-02-20'),
  (9, 7, '2026-02-25'),
  (11, 8, '2026-05-01'),
  (12, 8, '2026-05-01'),
  (10, 9, '2026-08-25'),
  (13, 9, '2026-08-25'),
  (14, 10, '2026-10-01'),
  (13, 10, '2026-10-01'),
  (11, 11, '2027-02-01'),
  (9, 12, '2027-02-15');
```

9.2 How the scripts were checked

No MySQL server was available while this solution was prepared, so the MySQL scripts were checked by translating them mechanically to SQLite (with foreign keys and CHECK constraints switched on) and running them with `tests/sqlite_smoke_test.py`. The test (a) creates all 31 tables, (b) loads all 310 rows, (c) runs `PRAGMA foreign_key_check`, (d) verifies seventeen cross-table business rules (some enforced by the MySQL triggers, the others guaranteed by the way objects are inserted; for example *every object has exactly one category row*, *a curator never exceeds his or her maximum*, *ON_LOAN if and only if a loan is open*), (e) confirms that twelve kinds of invalid data are **rejected** (bad identifier format, end date before start, duplicate living artist, a replica for an object not in the exhibition, deleting an object that has loan history, …) and (f) runs the thirteen transactions, whose results are shown in Appendix A. All of these checks pass.

The constructs that exist only in MySQL (`ENUM`, the generated column, the views written with `CREATE OR REPLACE`, the triggers and the `DELIMITER` command) were written to the MySQL 8.0 syntax but could not be executed in that test. If MySQL reports an error when you run `artsmuseum.sql`, it names the statement and line; the most likely causes are a server older than 8.0.16 (CHECK constraints are ignored but harmless) or a client that does not understand `DELIMITER` (run the trigger section in MySQL Workbench or with the `mysql` command-line client).

Task 10 – Top three lessons learnt

1. Most of the design is decided while reading, not drawing. The brief looked complete, yet careful reading changed the model more than any diagram did. The word "type" meant five different things; the museum lends objects out but only the borrowing side was specified; the artist's key included a death date that living artists do not have; and "status: on display" quietly duplicated the gallery an object hangs in. Each of these cost a sentence to fix in the requirements and would have cost a table redesign (and a data migration) if found during implementation. Next time we would write a one-line glossary entry for every noun and every repeated word in the brief *before* drawing anything, and re-check every transaction against the refined requirements after each change.

2. Decide deliberately where each rule is enforced, and learn what the DBMS cannot do. Declarative constraints are the strongest tool but each has a blind spot. A UNIQUE key treats NULLs as different, so the artist's identity needed a generated stand-in for "no death date". A CHECK constraint cannot see another table, so rules such as "a curator keeps to his or her specialisation and workload" needed triggers. MySQL does not fire triggers for rows changed by a cascading foreign key action, which is why a gallery cannot be deleted while objects hang in it (`RESTRICT`, not `SET NULL`). And no foreign key can say "every object must have a subclass row", so that rule lives in the insert transaction. Writing every business rule in a list and asking "which mechanism enforces this, and what can get around it?" made the integrity of the schema something we could argue for, not assume.

3. Test with realistic data, and make sure each test proves what it claims. Writing the sample data forced decisions the diagram had hidden (which statuses a borrowed object may have; that ON_LOAN must match an open loan; that a replica needs an exhibited object). Running the thirteen transactions on it showed immediately that, for example, an inner join in T9 would have hidden the exhibition with no invitations. We also learnt that a test can pass for the wrong reason: our first test of "duplicate living artist is rejected" did pass, but only because the inserted row had no key value; once the row was given a key, the test exercised the real UNIQUE rule. Testing both what must be accepted and what must be rejected, and checking *why* a statement failed, is what gives confidence in the design. Normalisation was the same: treated as a check applied to every relation (with the dependencies written down), not as a clean-up at the end, it removed the redundancy while the design was still cheap to change.

Appendix A – The transaction requirements as SQL

Each transaction is shown with the example values used in `04_transactions.sql` and the result obtained on the sample data (rows as returned by the database). T1 and T2 change data; T3 to T13 are queries. All results were produced on the freshly loaded sample data.

T1 – Add a new art object to the permanent collection

```
START TRANSACTION;
INSERT INTO ArtObject (objectID, title, description, yearCreated, category, ownership,
                       status, artistID, originID, epochID, styleID, galleryID)
VALUES ('AO0025', 'The Reading Room', 'Oil painting of readers in a sunlit library.', 1891,
        'PAINTING', 'PERMANENT', 'IN_STORE', 3, 4, 6, 4, NULL);
INSERT INTO Painting (objectID, paintingMedium, surfaceMaterialID)
VALUES ('AO0025', 'Oil', 1);
INSERT INTO PermanentObject (objectID, acquisitionDate, acquisitionCost)
VALUES ('AO0025', '2026-11-05', 61000.00);
COMMIT;
```

Result: one row is inserted into each of ArtObject, Painting and PermanentObject; the three inserts are committed together (they are rolled back together if any statement fails).

T2 – Change the end date of a featured exhibition

```
UPDATE FeaturedExhibition
SET    endDate = '2027-01-31'
WHERE  exhibitionID = (SELECT exhibitionID FROM Exhibition
                       WHERE exhibitionName = 'The Dutch Golden Age');
```

Result: one row updated. An end date earlier than the start date is refused by `ck_featured_dates`.

T3 – List all the borrowed art objects

```
SELECT ao.objectID, ao.title, ao.category, mu.museumName AS lender,
       b.dateBorrowed, b.dateDue, b.dateReturned,
       CASE WHEN b.dateReturned IS NULL THEN 'Held now' ELSE 'Returned' END AS loanState
FROM BorrowedObject bo
JOIN ArtObject ao ON ao.objectID = bo.objectID
JOIN Borrowing b  ON b.objectID  = bo.objectID
JOIN Museum mu    ON mu.museumID = b.museumID
ORDER BY ao.objectID, b.dateBorrowed;
```

Result on the sample data (7 rows):

objectID	title	category	lender	dateBorrowed	dateDue	dateReturned	loanState
AO0005	Portrait of a Merchant's Wife	PAINTING	Riverside Museum of Fine Art	2026-03-01	2027-03-01	NULL	Held now
AO0006	Winter Orchard	PAINTING	Tay Valley Art Museum	2025-02-01	2025-08-01	2025-07-28	Returned
AO0006	Winter Orchard	PAINTING	Tay Valley Art Museum	2026-05-01	2027-05-01	NULL	Held now
AO0011	Carved Temple Guardian	SCULPTURE	Highland Heritage Gallery	2026-01-15	2027-01-15	NULL	Held now
AO0016	Saint Michael and the Dragon	STATUE	Castlegate Sculpture Museum	2026-04-10	2027-04-10	NULL	Held now
AO0018	The Founder (Monument Maquette)	STATUE	Castlegate Sculpture Museum	2025-06-01	2026-06-01	2026-05-28	Returned
AO0020	Moonlit Ferry at Sumida	OTHER	Solway Print and Photography Museum	2026-09-01	2027-06-30	NULL	Held now

T4 – Count the art objects created by a given artist

```
SELECT a.artistName, a.dateBorn, COUNT(ao.objectID) AS numberOfObjects
FROM Artist a
LEFT JOIN ArtObject ao ON ao.artistID = a.artistID
WHERE a.artistName = 'Margaret Ellison' AND a.dateBorn = '1903-08-14'
GROUP BY a.artistID, a.artistName, a.dateBorn;
```

Result on the sample data (1 row):

artistName	dateBorn	numberOfObjects
Margaret Ellison	1903-08-14	3

T5 – Count the art objects per category displayed in the regular exhibition

```
SELECT ao.category, COUNT(*) AS numberOfObjects
FROM ArtObject ao
JOIN Gallery g           ON g.galleryID = ao.galleryID
JOIN RegularExhibition r ON r.exhibitionID = g.regularExhibitionID
GROUP BY ao.category
ORDER BY numberOfObjects DESC, ao.category;
```

Result on the sample data (4 rows):

category	numberOfObjects
PAINTING	5
STATUE	5
OTHER	4
SCULPTURE	3

T6 – List all the art objects borrowed from a given museum

```
SELECT ao.objectID, ao.title, b.dateBorrowed, b.dateDue, b.dateReturned
FROM Museum mu
JOIN Borrowing b  ON b.museumID = mu.museumID
JOIN ArtObject ao ON ao.objectID = b.objectID
WHERE mu.museumName = 'Tay Valley Art Museum'
ORDER BY b.dateBorrowed;
```

Result on the sample data (2 rows):

objectID	title	dateBorrowed	dateDue	dateReturned
AO0006	Winter Orchard	2025-02-01	2025-08-01	2025-07-28
AO0006	Winter Orchard	2026-05-01	2027-05-01	NULL

T7 – List the staff supervising a given featured exhibition

```
SELECT s.staffID, s.firstName, s.lastName, s.jobTitle, sf.allocatedOn
FROM SupervisesFeatured sf
JOIN Staff s ON s.staffID = sf.staffID
WHERE sf.exhibitionID = (SELECT exhibitionID FROM Exhibition
                         WHERE exhibitionName = 'Gods and Heroes in Marble')
ORDER BY s.lastName;
```

Result on the sample data (2 rows):

staffID	firstName	lastName	jobTitle	allocatedOn
12	Rhona	MacLeod	Gallery Supervisor	2026-05-01
11	Kwame	Mensah	Senior Gallery Supervisor	2026-05-01

T8 – List the curators responsible for maintaining a given art object

```
SELECT s.staffID, s.firstName, s.lastName, c.specialisation, m.assignedDate
FROM Maintains m
```

```
JOIN Curator c ON c.staffID = m.staffID
JOIN Staff s   ON s.staffID = c.staffID
WHERE m.objectID = 'AO0009'
ORDER BY s.lastName;
```

Result on the sample data (2 rows):

staffID	firstName	lastName	specialisation	assignedDate
3	Sofia	Lindqvist	SCULPTURE	2017-05-10
4	Daniel	Ruiz	SCULPTURE	2013-10-08

T9 – List all featured exhibitions with their dates and loyal visitors

```
SELECT e.exhibitionName, fe.startDate, fe.endDate,
       lv.loyaltyNo, CONCAT(lv.firstName, ' ', lv.lastName) AS visitorName
FROM FeaturedExhibition fe
JOIN Exhibition e          ON e.exhibitionID = fe.exhibitionID
LEFT JOIN Invitation i     ON i.exhibitionID = fe.exhibitionID
LEFT JOIN LoyalVisitor lv  ON lv.loyaltyNo   = i.loyaltyNo
ORDER BY fe.startDate, e.exhibitionName, lv.lastName;
```

Result on the sample data (12 rows):

exhibitionName	startDate	endDate	loyaltyNo	visitorName
Light and Water: The Impressionist Vision	2026-03-01	2026-06-30	LM1001	Catherine Bell
Light and Water: The Impressionist Vision	2026-03-01	2026-06-30	LM1007	Elena Rossi
Gods and Heroes in Marble	2026-05-15	2026-09-30	LM1006	Angus Murray
Gods and Heroes in Marble	2026-05-15	2026-09-30	LM1002	James Okoro
The Dutch Golden Age	2026-09-01	2026-12-31	LM1001	Catherine Bell
The Dutch Golden Age	2026-09-01	2026-12-31	LM1003	Sophie Laurent
The Dutch Golden Age	2026-09-01	2026-12-31	LM1008	David Walker
Modern Abstractions	2026-10-10	2027-01-31	LM1004	Ravi Patel
Modern Abstractions	2026-10-10	2027-01-31	LM1008	David Walker
Prints of the Floating World	2027-02-15	2027-05-30	LM1005	Mei Lin Chen
Prints of the Floating World	2027-02-15	2027-05-30	LM1004	Ravi Patel
Marchetti: A Baroque Life	2027-03-01	2027-07-31	NULL	NULL

T10 – List the art objects of a featured exhibition created by a given artist

```
SELECT ao.objectID, ao.title, ao.yearCreated
FROM ExhibitionObject eo
JOIN Exhibition e  ON e.exhibitionID = eo.exhibitionID
JOIN ArtObject ao  ON ao.objectID    = eo.objectID
JOIN Artist a      ON a.artistID     = ao.artistID
WHERE e.exhibitionName = 'The Dutch Golden Age'
  AND a.artistName = 'Pieter van der Linden' AND a.dateBorn = '1620-05-21'
ORDER BY ao.objectID;
```

Result on the sample data (2 rows):

objectID	title	yearCreated
AO0001	Harbour at Dawn	1655
AO0005	Portrait of a Merchant's Wife	1662

T11 – List the permanent objects that are lent out and not yet back (new)

```
SELECT ao.objectID, ao.title, mu.museumName AS borrower, l.dateLent, l.dateDue
```

```
FROM Lending l
JOIN ArtObject ao ON ao.objectID = l.objectID
JOIN Museum mu    ON mu.museumID = l.museumID
WHERE l.dateReturned IS NULL
ORDER BY l.dateDue;
```

Result on the sample data (2 rows):

objectID	title	borrower	dateLent	dateDue
AO0024	Lotus Pond at Night	Solway Print and Photography Museum	2026-10-01	2027-01-31
AO0012	Torso of Flight	Thistle Modern Art Centre	2026-09-15	2027-03-15

T12 – Replica income per featured exhibition (new)

```
SELECT e.exhibitionName, SUM(r.quantitySold) AS replicasSold,
       SUM(r.unitPrice * r.quantitySold) AS replicaIncome
FROM ReplicaSale r
JOIN Exhibition e ON e.exhibitionID = r.exhibitionID
GROUP BY e.exhibitionID, e.exhibitionName
ORDER BY replicaIncome DESC;
```

Result on the sample data (6 rows):

exhibitionName	replicasSold	replicaIncome
The Dutch Golden Age	246	9840
Gods and Heroes in Marble	98	9205
Light and Water: The Impressionist Vision	184	7320
Modern Abstractions	18	990
Prints of the Floating World	0	0
Marchetti: A Baroque Life	0	0

T13 – Whom to invite: visitors whose preferences match an exhibition (new)

```
SELECT DISTINCT lv.loyaltyNo, lv.firstName, lv.lastName, lv.email, lv.address
FROM FeaturedExhibition fe
JOIN Exhibition e          ON e.exhibitionID = fe.exhibitionID
JOIN ExhibitionFeature ef  ON ef.exhibitionID = fe.exhibitionID
JOIN VisitorPreference vp  ON vp.featureType = ef.featureType
                          AND vp.featureValue = ef.featureValue
JOIN LoyalVisitor lv       ON lv.loyaltyNo = vp.loyaltyNo
WHERE e.exhibitionName = 'Marchetti: A Baroque Life'
  AND NOT EXISTS (SELECT 1 FROM Invitation i
                  WHERE i.loyaltyNo = lv.loyaltyNo AND i.exhibitionID = fe.exhibitionID);
```

Result on the sample data (1 row):

loyaltyNo	firstName	lastName	email	address
LM1003	Sophie	Laurent	sophie.laurent@example.org	15 Ferryhill Road, Aberdeen AB11 6QN

Appendix B – References

- Connolly, T. and Begg, C. (2015) *Database Systems: A Practical Approach to Design, Implementation, and Management*. 6th edn. Harlow: Pearson. Used for the EER-to-relational mapping guidelines (generalisation, aggregation, association classes, multi-valued attributes) and for the definitions of 1NF, 2NF, 3NF and BCNF.
- Oracle (2024) *MySQL 8.0 Reference Manual*. https://dev.mysql.com/doc/refman/8.0/en/ Used for CHECK constraints, generated columns, FOREIGN KEY actions, triggers and the `ENUM` type.
- Object Management Group (2017) *Unified Modeling Language, version 2.5.1*. https://www.omg.org/spec/UML/2.5.1 Used for the class-diagram notation (generalisation, aggregation, composition, association classes, multiplicities).

- University of Aberdeen, School of Natural and Computing Sciences, *CS2019 Databases and Data Management* lecture and laboratory material (the DreamHome case study and the relation-schema notation used in Task 4).